

Analysis of the Problem

Description of Problem

The objective of this project is to develop a game called Flappy Bird. This endless bird game will involve a bird character soaring through dynamically generated vertical poles that contain gaps for the bird to pass through. The game will end if the player's bird collides with any of the poles or if the bird touches the water. Key features include a straightforward text-based input system for capturing player name and age, with real-time validation to ensure accurate data entry and also includes player data storage and progressional tracking.

The target users for this game are individuals who can operate a computer, making the interface intuitive and accessible for players of various ages. The game will integrate with a SQL database to store individual player data such as player name, age, high score, retries, and level achieved.

This project aligns with the Advanced Higher Computing Science requirements because it incorporates a user-friendly interface and validates inputs, ensuring robustness. Additionally, a bubble sort algorithm will be used to sort the scores, allowing efficient organization of high scores and to show the highest score achieved by the player, adding a competitive element for players.

Scope

The scope of my project will include:

1. A completed design, data dictionary, query design, wireframes showing the game UI and UML case diagram to show the details of the users and their interaction with the game development.
2. A working game with fully functional database to record data of individual players
3. Test plans with extreme, exceptional, and normal test data to handle any unexpected inputs. Description of test cases and the expected outputs of my program.
4. Implementation of a bubble sort algorithm to organize scores efficiently, displaying the highest score
5. The software specification and the results of final testing.
6. An evaluation report.

Constraints

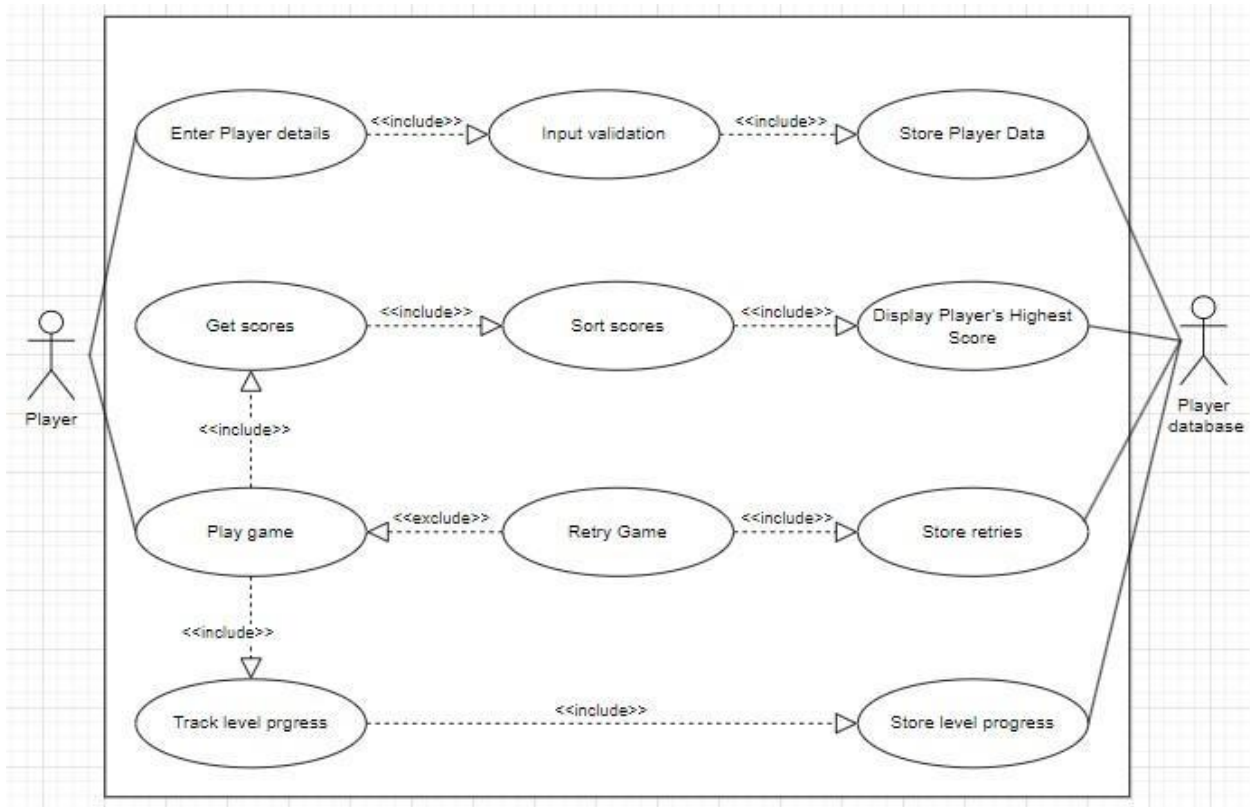
The technical, time and economic constraints are listed below:

1. The game will be coded in python as I have the most experience working with python
2. Pygame is imported into my project because I cannot create a game using only the Python language.
3. MySQLconnector is also imported in my project to connect my game to the database and execute queries via my source code.
4. There will be no economic costs involved for the development of my project because I am using visual studio to code my project which is an open-source free to use software
6. The project also relies on freely available libraries and tools, avoiding any additional expenses.
5. I will ensure that my project is completed by the deadline of 29th March as it will need to be delivered to SQA for marking. The submission deadline is 4th April 2019

Boundaries

1. The system will prompt the player for their name and age once per session. The player has unlimited retries if they fail the initial input attempt.
2. The age must also be less than or equal to 120 to prevent errors due to unrealistic inputs.
3. The age must have numeric characters only and must be equal to or greater than 8.
4. The email must have an "@" and "." in it for input validation
5. The player name input should have a maximum limit (e.g., 20 characters) to avoid overly long names that may cause display issues in the database

UML Use Case Diagram



Requirement Specification

Flappy Bird is a game created for users who love a challenging and exciting game that has a lot of competitive aspects to it .The game aims to entertain players by challenging them to achieve high scores which is sent to a database to compare other high scores of different players further enhancing the competitive aspect.

Functional requirements

- 1.The game must allow the bird to fall naturally due to gravity unless the player presses the mouse button or the spacebar to make it ascend.
- 2.Display the bird, poles, and ground.
- 3.Animate the bird's flapping and falling.
- 4.The game will provide a main menu and a game-over screen.
- 5.The game must generate poles with a fixed gap at regular intervals.
- 6.After a collision is detected the game must show the death screen with the “Retry” button.
- 7.The game constantly checks for collisions between player and the vertical poles

during the game to display the game-over phase.

8. The game asks for player's details such as player name, email, age. The game records high scores, retries and level for each individual player.

9. The game will validate keyboard inputs, which are the player's name, email, and age.

10. The current score will be displayed during the gameplay, along with the level and speed of the obstacles thus allowing players to track their progress.

11. The game tracks the user's score everytime the user dies the score is stored in the scores array.

12. The game will implement a "partial" bubble sort algorithm on the scores array and the highest score is displayed using the "partially" sorted score's array. (Partial bubblesort explained later in evaluation).

13. The game will allow players to retry when they collide with the vertical poles or touch the ground in-game.

14. As the game progresses, the speed of the vertical poles shall increase up to a predefined maximum speed limit.

15. The game must display the game menu with instructions and with the "Start" button.

16. The game must connect to a player database hosted on your pc.

17. A query is created to generate the player details table.

18. A SELECT query is created to find matching records of an existing player to modify his details for update query

19. A UPDATE query is created to update the highscores of the player if his current highscore is greater than his previous highscore

20. A INSERT query is created to insert the player details if player records do not exist.

End-User requirements

I have gathered information on the aspects that the end users want in the flappy bird game which is listed below:

1. Inputs that are taken must have input validation, should provide clear instructions, validation messages for empty or incorrect fields.

2. The user-interface must be clear enough to understand the controls and mechanics without needing detailed instructions.

3. The game should become progressively challenging over time.

4. Players should be able to see their current score and high score, allowing them to track their progress and set goals for improvement.

5. Players should have the option to retry the game immediately if they collide with an obstacle or fall in water.

Inputs,Processes and Outputs

Inputs:

Player Name

Age

Email

[Player key inputs] **Processes:**

1.Players sign up or login

- 1.Name input must have no special characters, at least 4 characters, and a maximum of 20 characters.
- 2.Age input must be a number from 8 to 120 and must contain only a number .
- 3.Both name and age must not be left blank.
4. Email validation must be rigid including checking for empty spaces and presence of “@” and “.” etc.

2.Gameplay

- 1.Display the bird with the start screen and look for player input to start the game.
- 2.Detect collision between the bird and poles or water to end the game
- 3.Update and display the score as the bird moves and avoid the obstacles.

3.Collision

- 1.If an collision is found the game ends immediately and add the score to the scores array.
- 2.The scores array is sorted out using bubble sort in descending order.
- 3.Since the score is sorted the highest score is the first element which is extracted and displayed as the high score in the death screen.

4.Retry Prompt

- 1.The player can retry if needed and the number of retries will also be stored.
- 2.If the player decides to retry and the game starts as it was in its initial state which includes the obstacles speed and the score

5.Database Interaction

- 1.After the collision has occurred between player and the poles the player data is sent to the database.
- 2.If the player ends the pygame session the highest score is updated in the database for the individual person.
- 3.If the same person starts a new session with the same player name,age and email their scores and high score is just updated instead of creating a new record.

Outputs:

- 1.Real-time score,Level and display during gameplay.
- 2.Retry prompt with options after a collision.
- 3.Immediate feedback for invalid inputs.
- 4.Displays options after a collision to either retry or exit.
- 5.Bird, Poles and water(ground).

Project Plan

The list of tasks with their allocation time is shown below:

Task	Sub Task	Duration	Start Date	Target Date
Analysis	Write the description of the project	1 day	1st Nov	1st Nov
	Write the scope and Constraints of the project	2 days	2nd Nov	3rd Nov
	Write the Boundaries of the project	1 day	3th Nov	3th Nov
	Create an UML case diagram for the project	1 day	4th Nov	4th Nov
	Create the requirement specification and write the functional requirement.	½ day	5th Nov	5th Nov
	Write the End user requirement	½ day	6th Nov	6th Nov
	Create the project Plan for the Project	¼ day	7th Nov	7th Nov

Design	Create wireframes for user interface	2 days	18th Nov	20th Nov
Task	Sub Task	Duration	Start Date	Target Date
	Create UML class diagram	1 day	20th Nov	21th Nov
	Pseudocode	6 days	1th Dec	6th Dec
	Query Design	1 day	7th Dec	7th Dec
	Data dictionary	1 day	10th Dec	10th Dec
Implemen- tation	Creating User Interface	3 days	13th Dec	16th Dec
	Creating Bird class and Player logic	5 days	16th Dec	22th Dec
	Create Pole class and sorting logic	3 days	23th Dec	26th Dec
	Create overall logic including collision,start and retry buttons	8 days	26th Dec	5th Jan
	Create database and query logic in SQL and integrate to python	5 days	5th Jan	11th Jan
	Complete documentation and log of testing	3 days	11th Jan	14th Jan
	Comprehensive test plan	4 days	27th Jan	1st Feb

Testing	Complete persona and test case	½ day	2nd Feb	2nd Feb
	Implement test plan and write results of testing	4days	3rd Feb	6th Feb
Evaluation				

Resources Required

The resources needed for each stage of development is shown below :

Analysis	Google docs online Google Chrome Microsoft Planner
Design	Google docs online Scholar(Google Chrome) Draw.io (UML Case Diagram) Paint (User Interface)
Implementation	Visual Studio 2022/Visual Studio Code XAMPP Control Panel (SQL and Apache for database) Google Chrome Google docs online
Testing	Visual Studio Code/Visual Studio 2022 Google Chrome XAMPP Control Panel (SQL and Apache for database) Google docs online
Evaluation	

Design of the Solution

User-Interface Design

Input validation interface

Input validation prompt

Enter your name:

Enter your email:

Enter your age:

When name is entered input validation is carried out i.e.it shouldn't be too long etc..

When email is entered input validation i.e. the email should contain "@" and "." character and the dot must be after the "@" character

When age is entered the input validation i.e. age must be a numeric character that should be greater than or equal to 8 and less than or equal to 120

Game-menu Interface

Flappy Bird



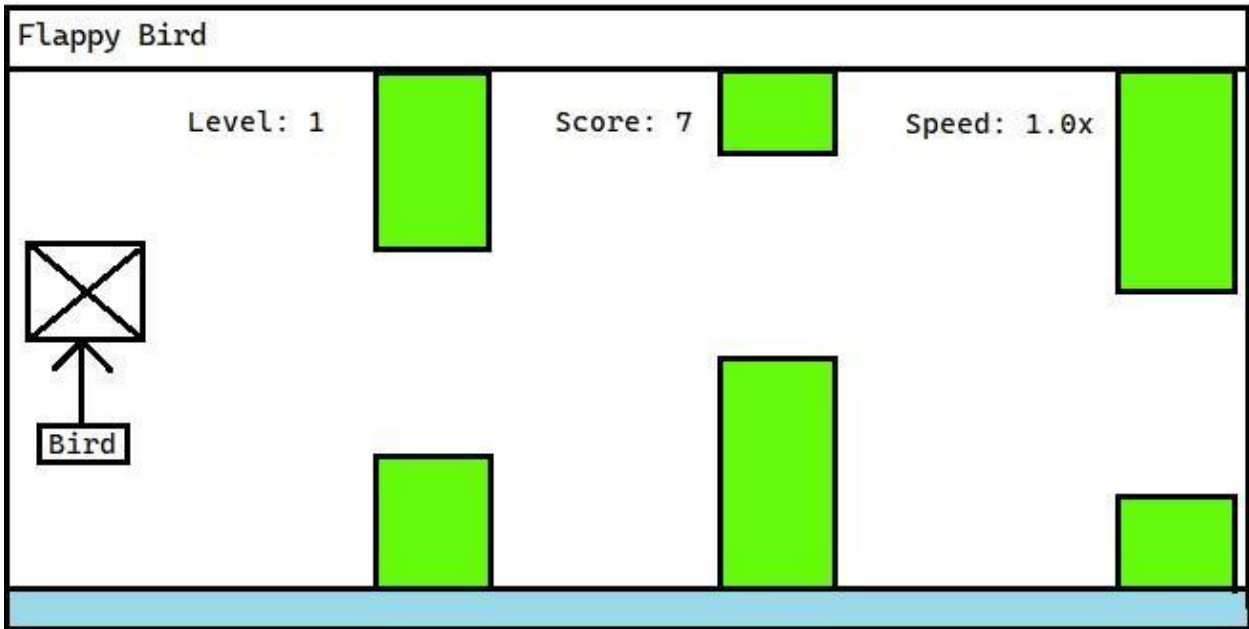
Bird

Press SPACEBAR or Click the button to Start

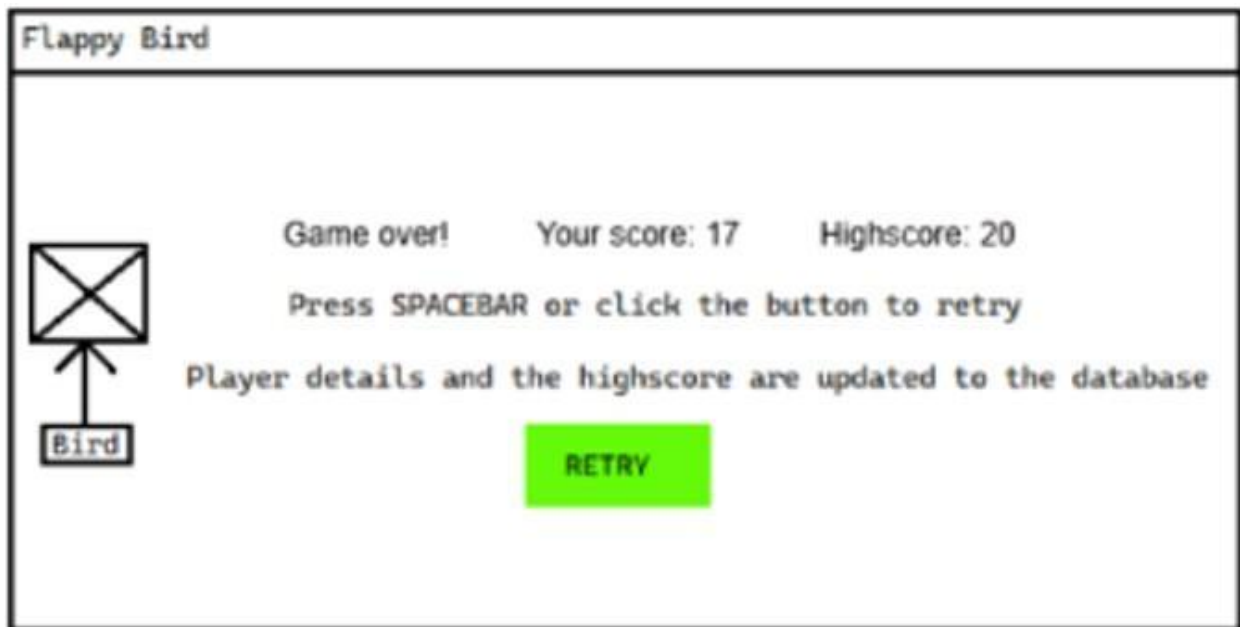
After Starting, use SPACEBAR or CLICK to flap!

START

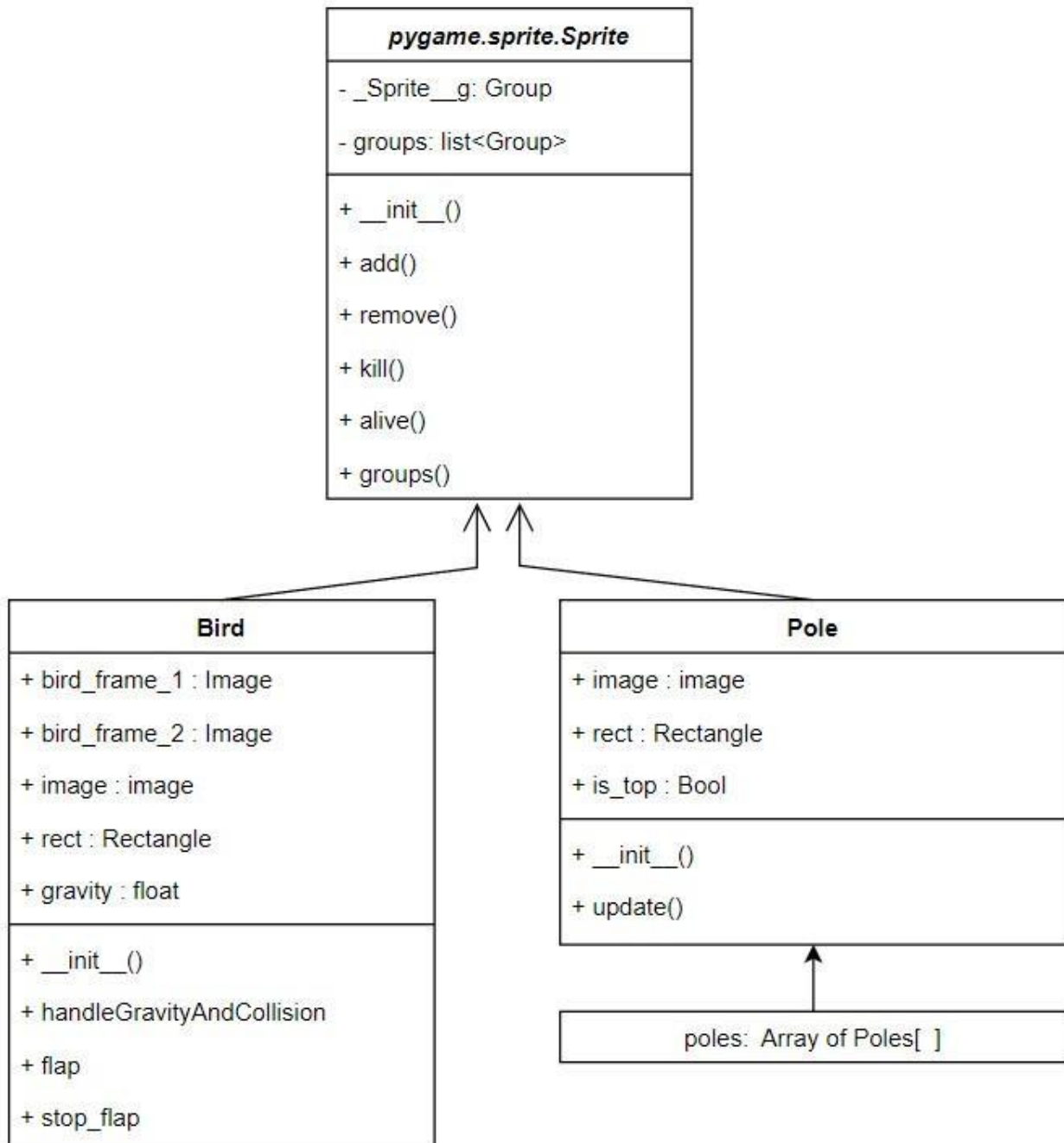
Game-active Interface



Game-over/Retry Interface



UML Class Diagram



The `pygame.sprite.Sprite` class is an inbuilt class in the `pygame` library designed for objects with attributes like `Image` and `rect` (`Rectangle`). The `Bird` and `Pole` subclasses inherit from the parent class `pygame.sprite.Sprite` to use the attributes.

Pseudocode

#This Function that checks for input validation on name,email and age_input
FUNCTION `Input_validation()`:

```

WHILE True:
    name = INPUT("Enter your name: ")
    TRIM name BY REMOVING STARTING AND LEADING TRAILS
    IF length(name) >= 20:
        SEND "Name must be less than 20 characters TO DISPLAY"
        SKIP ITERATION
    IF name IS BLANK:
        SEND "Name cannot be blank" TO DISPLAY
        SKIP ITERATION
    email = INPUT("Enter your email: ")
    TRIM email BY REMOVING STARTING AND LEADING TRAILS
    IF "@" is not in email OR "." is not in the part of email after "@":
        SEND "Please enter a valid email address" TO DISPLAY
        SKIP ITERATION
    ELSE:
        local_part = EXTRACT the words before "@"
        domain = EXTRACT the words after "@"
        IF "." not in domain or
        domain starts with "." or ends with ".":
            SEND "Please enter a
            valid email address" TO DISPLAY
            SKIP ITERATION
        IF local_part is empty or " " in local_part:
            SEND "Please enter a
            valid email address" TO DISPLAY
            SKIP ITERATION
        FOR checkempty IN [Extract words before and after "." in domain]:
            IF checkempty IS EQUAL TO ""
            SEND "Please enter a valid email address" TO DISPLAY

```

```

                                SKIP      ITERATION
        age_input    =    INPUT("Enter    your    age:    ")
    TRIM age_input BY REMOVING STARTING AND LEADING TRAILS
        IF    age    is    not    a    digit:
            SEND "Age must be a valid number" TO DISPLAY
                                SKIP      ITERATION
        AGE = CONVERT age FROM STRING TO INTEGER
    IF age < 8 or age > 120:
        SEND "Age must be at greater than or equal to 8 but less than or equal to
        120" TO DISPLAY
        SKIP ITERATION
    RETURN name,email,age
END WHILE LOOP
END FUNCTION

```

#Function that sorts the scores array when the player gets a new score

FUNCTION Partial_BubbleSort(scores):

#This function has "partial" bubble sort because unlike regular bubble sort it doesn't sort the whole array but performs a single pass of Bubble sort to move the highest score to the front. This design is intentional because after the player dies, the score is moved to where it's required rather than sorting the whole array.

swapped = True

n = length(scores)

WHILE swapped == True:

swapped = False

FOR LOOP i FROM 0 TO n-1:

IF scores[i] < scores[i+1]:

```
SET scores[i],scores[i+1] TO scores[i+1],scores[i]
```

```
RETURN scores #The score is returned early as it just sorts a single pass
```

```
END FUNCTION
```

```
name,age,email = CALL METHOD input_validation()
```

```
IMPORT pygame
```

```
IMPORT mysql.connector
```

```
random
```

```
IMPORT
```

```
#This function establishes a connection to the local database server and creates  
a database if it doesn't exist
```

```
FUNCTION create_connection():
```

```
SET mydatabase,mycursor AS GLOBAL VARIABLES
```

```
SET mydatabase = mysql.connect(
```

```
host = "localhost",
```

```
user = "root"
```

```
password = ""
```

```
)
```

```
DECLARE CURSOR BY mydatabase.cursor TO UPDATE AND EDIT RECORDS
```

```
mycursor.execute query("CREATE DATABASE IF NOT EXISTS Playerdatabase")
```

```
NAME OF mydatabase IS "Playerdatabase"
```

```
mycursor.execute QUERY FOR INITIALIZATION( ""
```

```
CREATE TABLE IF NOT EXISTS playerdetails(
```

```
player_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
name VARCHAR(255),
```

```
age INT,  
email VARCHAR(255),  
highscore INT,  
level INT,  
retries INT  
""")
```

```
END FUNCTION
```

```
CALL METHOD create_connection()
```

#This function Updates or inserts player data everytime the player dies

```
FUNCTION update_or_insert_records(name,age,email,highscore,level,retries):
```

```
    query = "SELECT player_id,highscore FROM playerdetails WHERE name = %s
```

```
    AND age = %s AND email = %s"
```

```
    EXECUTE THE QUERY ABOVE WITH ESCAPE VALUES(query,(name,age,email))
```

```
    FETCH THE RESULT ROW BY result = mycursor.fetchone()
```

```
    IF result HAS ONE RESULT DO :
```

```
        player[id] = FIRST ELEMENT OF THE TUPLE result
```

```
        previous_highscore = SECOND ELEMENT OF THE TUPLE result
```

```
        IF highscore > previous_highscore:
```

```
            update_query = "UPDATE playerdetails SET highscore = %s,level =
```

```
            %s,retries = %s WHERE player_id = %s"
```

```
EXECUTE THE QUERY ABOVE WITH ESCAPE VALUES TO PREVENT
```

```
    SQL INJECTION(query,(highscore,level,retries,player_id))
```

```
ELSE :
```

```
    insert_query = "INSERT INTO playerdetails
```

```

(name,age,email,highscore,level,retries)
VALUES (%s,%s,%s,%s,%s,%s)"

EXECUTE THE QUERY ABOVE WITH ESCAPE VALUES TO PREVENT

SQL INJECTION(insert_query,(name,age,email,highscore,level,retries))
mydatabase.commit() #Saves all changes made to the database

```

#Bird class represents the player so it handles the appearance,movement and collision mechanics of the player. The bird or the player “character” can be controlled using this class.

```

CLASS Bird INHERITS pygame.sprite.Sprite:
    CONSTRUCTOR __init__(SELF):    #Constructor for Bird class
        CALL      SUPER      CONSTRUCTOR
    DECLARE SELF.bird_frame_1 INITIALLY "flapping image"
    DECLARE SELF.bird_frame_2 INITIALLY "normal image"
    DECLARE SELF.image INITIALLY bird_frame_2
    DECLARE SELF.birdrect TO image.create_rectangle(position x,y)
    DECLARE SELF.gravity INITIALLY 0
END CONSTRUCTOR

```

#This function implements gravity on the bird and checks for collision with the ground if a collision is detected true is returned else false

```

FUNCTION      handleGravityAndCollision(SELF):
    IF      game_active      is      True:
    INCREMENT SELF.gravity BY 0.35
    INCREMENT SELF.rect.y BY SELF.gravity

```



```

        IF SELF.rect.bottom >= 420:
            SET SELF.rect.bottom TO 420
            RETURN True
        RETURN False
    END FUNCTION

```

```

FUNCTION flap(SELF): #This function contains code to allow the bird to flap
    IF game_active is True:
        SET SELF.gravity = -5.75
        SET self.image TO SELF.bird_frame_1  #(Flapping Image)
    END FUNCTION

```

```

FUNCTION stop_flap(SELF): #This function contains code to reset flap image
    SET SELF.image TO SELF.bird_frame_2  #(Normal image)
    END FUNCTION

```

```

END CLASS

```

#Pole class is responsible for npc (non-player controlled) “characters” which are the poles that the bird has to escape in order to survive. This class is responsible for movement, collision and appearance.

CLASS Pole INHERITS pygame.sprite.Sprite:

```

    CONSTRUCTOR __init__(SELF,x,y,Is_Top): #Constructor for Pole class

```

```

        CALL SUPER CONSTRUCTOR

```

```

    DECLARE SELF.image INITIALLY “Pole_bottom” #Sets image for bottom pole

```

```

        IF Is_Top == True:

```

```

            SET SELF.image TO “Pole_Top” #Sets image for top pole

```

```

        CREATE RECTANGLE AS SELF.rect WITH POSITION x,y FOR THE Bird image

```

```
        SET SELF.speed = 5 AS INITIAL POLE SPEED
    CONSTRUCTOR
```

END

```
    # This function updates the position of the pole and removes it from the game  
    if it moves off-screen.
```

```
FUNCTION update(SELF):
```

```
    IF game_active == True:
```

```
        #As the game progresses the speed multiplier and speed is increased
```

```
            SELF.rect.x -= 5 * speed_multiplier
```

```
            IF SELF.rect.right < 0:
```

```
                SELF.kill() #Pole is removed if it goes out of the screen
```

```
    END FUNCTION
```

```
END CLASS
```

```
INITIATE PYGAME
```

```
SCREEN = SET SCREEN DISPLAY(900,450) #900 pixels horizontal and 450 vertical
```

```
SET GAME NAME AS 'Flappy Bird'
```

```
SET clock TO create_clock()
```

```
DECLARE water_surf INITIALLY "Water" #Water image
```

```
#The single bird group is used for player character
```

```
CREATE bird GROUP AS SPRITE.GROUPSINGLE()
```

```
ADD Bird CLASS INSTANCE TO THE SINGLE bird GROUP
```

```
#This array contains a list of pole objects to spawn infinite instances. Because
```

using a "Group" wouldn't meet the project requirements.

```
object_list = []
```

```
scores= [] #The scores array captures the player's score upon death
```

#This constant determines the gap size between the top and bottom poles

```
SET GAP_SIZE = 600
```

#This constant determines where the pole starts from in the screen

```
SET POLE_X_START = 950
```

**#This constant determines the minimum offset from the centre of the screen
where the pole can go**

```
SET POLE_Y_MIN = 75
```

#This constant determines the maximum offset from the centre of the screen where the pole can go

SET POLE_Y_MAX = 300

SET game_active = False **#This variable defines the game active state**

SET game_over = False **#This variable defines the game over state**

SET score_reset_time = 0 **#Tracks the start time for the score calculation**

SET level = 1 **#This variable keeps track of player level and updates it**

SET retries = 0 **#This variable counts the number of retries the player makes**

SET speed_multiplier = 1 **#This variable keeps track of the pole speed**

SET next_level_score = 10 **#This variable keeps count of the next score needed to increase the level and speed_multiplier**

SET score_recorded = False **#This variable ensures that player's score is recorded only once per game session**

SET sorted = False **#This variable ensures score is sorted once**

SET highscore = 0 **#This variable stores the highscore**

#This function creates top and bottom poles with a random gap, adjusts their positions, and adds them to the obstacle list.

FUNCTION create_poles():

Gap_y = GENERATE A RANDOM NUMBER BETWEEN POLE_Y_MIN AND
POLE_Y_MAX

CREATE top_pole WITH THE FOLLOWING CHARACTERISTICS:

X-COORDINATE = POLE_X_START

Y-COORDINATE = gap_y - (GAP_SIZE / 2) - 330 **#The 300 constant is only**

for the top pole because for the top pole the image is bigger to prevent players

from going over the poles and storing invalid scores

CREATE bottom_pole WITH THE FOLLOWING CHARACTERISTICS:

X-COORDINATE = POLE_X_START

Y-COORDINATE = gap_y + (GAP_SIZE / 2)

```

        APPEND top_pole TO obstacle_list
    APPEND bottom_pole TO obstacle_list
END FUNCTION

```

#This function dynamically updates the scores with intended logic and displays it

```
FUNCTION update_and_display_score():
```

```
    #pygame.time gives the time since the game started in ms so the score is
    reset using score_reset_time,set when the game starts
```

```
    SET score = int(pygame.time.get_ticks() / 1000) - score_reset_time
```

```
    CREATE score_surf AS RENDERED TEXT USING font WITH THE STRING
```

```
    `Score:
```

```
    {score}` IN BLACK COLOR
```

```
    CREATE score_rect ,THE RECTANGLE OF score_surf WITH CENTER AT (450, 20)
```

```
    DRAW score_surf ON screen AT score_rect
```

```
    RETURN score
```

```
END FUNCTION
```

#This function displays the level and speed_multiplier during the game

```
FUNCTION display_level_and_speed():
```

```
    CREATE level_surf AS RENDERED TEXT USING font WITH THE STRING
```

```
    `Level:
```

```
    {level}` IN BLACK COLOR
```

```
    CREATE level_rect , THE RECTANGLE OF level_surf AND CENTER AT (150,20)
```

```
    DRAW level_surf ON screen AT level_rect
```

```
    CREATE speed_surf AS RENDERED TEXT USING font WITH THE STRING
```

```
    `Speed: {speed_multiplier: }` IN BLACK COLOR
```

```
    CREATE speed_rect, THE RECTANGLE OF speed_surf WITH CENTER AT(750,20)
```

```
DRAW speed_surf ON screen AT speed_rect
END FUNCTION
```

```
font_large = DEFAULT FONT WITH SIZE 50 IS CREATED FOR LARGE TEXT
font = DEFAULT FONT WITH SIZE 35 IS CREATED FOR TEXT
```

#Creation of the start button

```
CREATE start_button_rect position = (350,200),width = 200 and height = 80
button_text IS CREATED WITH TEXT 'START' no anti-aliasing AND BLACK
COLOUR
```

#Creation of the retry button

```
CREATE retry_button_rect position = (390,300),width = 120 and height = 60
button2_text IS CREATED WITH TEXT 'RETRY' no anti-aliasing AND BLACK COLOUR
```

```
WHILE True: #Main game loop for handling events, updates, and rendering
    FILL THE SCREEN WITH THE COLOUR WHITE
    KEEP bird DRAWN ON screen ALWAYS
```

```
FOR each event IN PYGAME EVENT QUEUE: #loops through events in pygame
```

```
    IF event.type IS QUIT:
```

```
        EXIT THE GAME
```

```
IF game_active == False AND game_over == False:
```

```
    #The game can be started either by mousebutton press on start button or  
by pressing spacebar
```

```
    IF event.type IS mousebuttondown PRESS:
```

```
        IF COLLISION OCCURS WITH start_button_rect BY mousebuttondown:
```

```
            SET game_active TO TRUE
```

```

    #score_reset_time resets the score at the start of the game

    score_reset_time = int(pygame.time.get_ticks() / 1000)

    #The timer below is for the pole obstacle which is defined later

    SET A TIMER TO TRIGGER custom_event EVERY 1200 milliseconds

    IF event.type IS keyboard PRESS:
    IF BUTTON THAT IS PRESSED IS spacebar:
        SET game_active TO TRUE
    score_reset_time = int(pygame.time.get_ticks() / 1000)

    SET A TIMER TO TRIGGER custom_event EVERY 1600 milliseconds

    #The bird can be flapped by either spacebar or mousebutton press

    IF gameactive == True and event.type IS keyboard PRESS:
        IF BUTTON THAT IS PRESSED IS spacebar:
            CALL METHOD flap OF Bird SPRITE USING Bird.sprite.flap()
    IF gameactive == True and event.type IS keyboard PRESS RELEASE:
        IF BUTTON THAT IS RELEASED IS spacebar:
            CALL METHOD stop_flap OF Bird SPRITE USING Bird.sprite.stop_flap()
    IF gameactive == True and event.type IS mousebuttonmousedown:
        CALL METHOD flap OF Bird SPRITE USING Bird.sprite.flap()
    IF gameactive == True and event.type IS mousebuttonmouseup:
        CALL METHOD stop_flap OF Bird SPRITE USING Bird.sprite.stop_flap()

    IF game_over == True:

    #The game can be restarted either by mousebutton press on retry button or

```

by pressing spacebar.

```
                IF event.type IS mousebuttondown PRESS:
IF COLLISION OCCURS WITH retry_button_rect BY mousebuttondown:
                                SET game_active TO TRUE
                SET game_over TO FALSE
RESET bird POSITION TO RESTART THE GAME
RESET bird_gravity TO 0 WHEN RESTARTING
SET score_recorded TO FALSE TO COUNT THE NEXT SCORE
CLEAR obstacle_list TO REMOVE POLES FOR A FRESH RESTART
#Score time is reset for the next instance of score to be recorded accurately
                score_reset_time = int(pygame.time.get_ticks() / 1000)
                SET level TO 1 TO RESET LEVEL AFTER RETRYING THE GAME
                SET speed_multiplier TO 1 TO RESET BIRD SPEED WHILE RETRY
```



```

INCREMENT retries BY 1 WHEN RETRY BUTTON IS PRESSED
#Next level score is reset to update speed and level accurately
                                SET    next_level_score    TO    10
                                IF      event.type    IS      keyboard    PRESS:
IF BUTTON THAT IS PRESSED IS spacebar:
    SET game_active TO TRUE
                                SET    game_over    TO    FALSE
RESET bird POSITION TO RESTART THE GAME
RESET bird_gravity TO 0 WHEN RESTARTING
SET score_recorded TO FALSE TO COUNT THE NEXT SCORE
CLEAR obstacle_list TO REMOVE POLES FOR A FRESH RESTART
                                score_reset_time = int(pygame.time.get_ticks() / 1000)
                                SET level TO 0 TO RESET LEVEL AFTER RETRYING THE GAME
                                SET speed_multiplier TO 1 TO RESET BIRD SPEED WHILE RETRY
                                INCREMENT retries BY 1 WHEN RETRY BUTTON IS PRESSED
                                SET    next_level_score    TO    10
IF game_active == TRUE and event.type == TIMER (CREATED EARLIER)
    CALL method create_poles() #To add poles when the game is active
END FOR LOOP

IF game_active == False and game_over == False: #Display for game menu
    DRAW a green rectangle on the screen at the position of start_button_rect

```

DRAW button_text ALONGSIDE start_button_rect **#button_text is "Start"**

start_text IS CREATED WITH TEXT "Press SPACEBAR or Click the button to

Start" WITH NO ANTI-ALIASING AND BLACK COLOR

CREATE start_rect, THE RECTANGLE OF speed_text AT POSITION(450,100)

DRAW start_text ON screen AT start_rect

start1_text IS CREATED WITH TEXT "After Starting, use SPACEBAR or CLICK to flap!" WITH NO ANTI-ALIASING AND BLACK COLOR

CREATE start1_rect, THE RECTANGLE OF start1_text AT POSITION(435,150)

DRAW start1_text ON screen AT start1_rect

IF game_active == True:

#The for loop updates each pole, draws it on screen, and ends the game if collision with the bird occurs.

FOR EACH pole IN obstacle_list():

CALL update() method of pole for handling pole's moment i.e.speed etc

DRAW pole.image ON screen AT pole.rect

IF COLLISION OCCURS BETWEEN THE bird AND pole:

game_active = False **#Collision Mechanics**

game_over = True

BREAK

score = CALL METHOD

update_and_display_score()

```

#Whenever the score is greater than the next_level_score the level and
speed_multiplier is increased
IF score IS
GREATER THAN OR EQUAL TO next_level_score :
    level +=1
    next_level_score += 10
    IF speed_multiplier IS LESS THAN 2
        speed_multiplier += 0.1
CALL METHOD display_level_and_speed()

#handleGravityAndCollision method takes care of bird gravity and returns
true if the bird collides with the ground
IF METHOD handleGravityAndCollision() == TRUE:
    game_active = False
    game_over = True

IF gameover == True:

#score should be recorded only once and to do that score_recorded exists
IF score_recorded == False:
    ADD score TO scores ARRAY
    SET score_recorded TO True

#score should be recorded only once and to do that score_recorded exists
IF sorted == False:

```

SET scores AS RETURN VALUE OF METHOD

Partial_BubbleSort(scores) IF scores HAS ATLEAST 1 VALUE:

SET highscore TO FIRST ELEMENT OF scores ARRAY

#Next few lines of code is focused on the UI of the game

DRAW a green rectangle on the screen at the position of start_button_rect

DRAW button2_text ALONGSIDE start_button_rect **#button2_text is “Retry”**

game_over_text IS CREATED WITH TEXT “Game Over! Your score:{score

variable} Highscore: {highscore variable} ”

CREATE game_over_rect, THE RECTANGLE OF game_over_text AT
POSITION(450,150)

DRAW game_over_text ON screen AT game_over_rect

retry_over_text IS CREATED WITH TEXT “Press SPACEBAR or click the
button

to retry”

CREATE retry_over_rect, THE RECTANGLE OF retry_over_text AT
POSITION(450,200)

DRAW retry_over_text ON screen AT retry_over_rect

CALL METHOD **#Updates or inserts player details into the player database**

update_or_inserts_records(name,age,email,highscore,level,retries)

#The next few lines of code is focused on UI again

```

database_updation_text IS CREATED WITH TEXT "Player details and the score
are updated to the database"

CREATE database_updation_rect, THE RECTANGLE OF
        database_updation_text    AT    POSITION(450,250)

DRAW database_updation_text ON screen AT database_updation_rect
DISPLAY water_surf at position (0, 420) on screen    #water image

        UPDATE    the    screen    for    each    frame

        WAIT for the next frame (limit to 60 frames per second)
END WHILE

```

Design of Integration

Query for database setup

The database integration setup is based on MySQL/MariaDB SQL syntax, which differs from Access SQL. Below are the queries used for this setup:

Query 1:

```
CREATE DATABASE IF NOT EXISTS Playerdatabase
```

Query 2:

```
CREATE TABLE IF NOT EXISTS playerdetails (
        id INT AUTO_INCREMENT PRIMARY KEY, #Surrogate key for the database
#varchar is a data type that stores variable-length text with a set maximum size.

```

```
        name    VARCHAR(255),  
age INT, #int is integer data type  
        email    VARCHAR(255),  
        highscore    INT,  
level INT,  
retries INT  
)
```

Query design for Player Database to update/insert records

Query 3:

This query first checks the database if there is an person with the same name,age,email and then gets the result and works accordingly.

Field(s)/ calculation(s)	id,highscore
Table(s) query(- ies)	playerdetails
Search criteria	name = {name variable},age ={age variable},email = {email variable}

Query 4a:

The following query runs if the first query 1 finds a matching record.The player_id and previous highscore from the record is retrieved. If the current highscore is greater than the previous highscore,an update query is executed to store the new highscore.

Update table(s)	playerdetails
Set	highscore = {highscore variable}
Search criteria	id = {player_id variable}

Query 5b:

The following query runs if query 1 does not find a matching record.The query creates a record using insert and whenever the update or insert record function is called again, the records of the matching user are updated again using query 2a.

Insert Into Table(s)	playerdetails
Values	name = {name variable},age = {age variable},email = {email variable},highscore = {highscore variable},level = {level variable },retries = {retries variable}

Data dictionary

To be noted PK = primary key and SK = surrogate key

Entity: playerdetails

Attribute Name	Key	Type	Size	Required	Validation
player_id	PK/SK	integer		Yes	Auto increment
name		varchar	20	Yes	
email		varchar		Yes	
age		integer		Yes	
highscore		integer		Yes	
level		integer		Yes	
retries		integer		Yes	

Implementation of the Problem

Screenshots of user interface

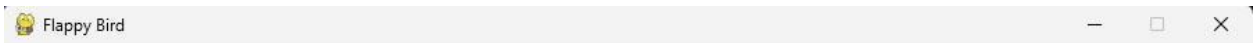
Input Validation

```

PS C:\Users\Namis\Desktop\Flappy Bird> & C:/Users/Namis/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/Namis/Desktop/Flappy Bird/Working.py"
Enter your name:
Name cannot be blank
Enter your name: Test Case Incorrect 12345
Name must be less than or equal 20 characters
Enter your name: Test Case Working
Enter your email: TestCaseIncorrect@gmailcom
Please enter a valid email address
Enter your name: Test Case Working
Enter your email: TestCaseWorking@gmail.com
Enter your age: 2000
Age must be at greater than or equal to 8 but less than or equal to 120
Enter your name: Test Case Working
Enter your email: TestCaseWorking@gmail.com
Enter your age: 17
Validation successful launching game...
pygame 2.6.1 (SDL 2.28.4, Python 3.13.1)
Hello from the pygame community. https://www.pygame.org/contribute.html

```

Game Menu Interface

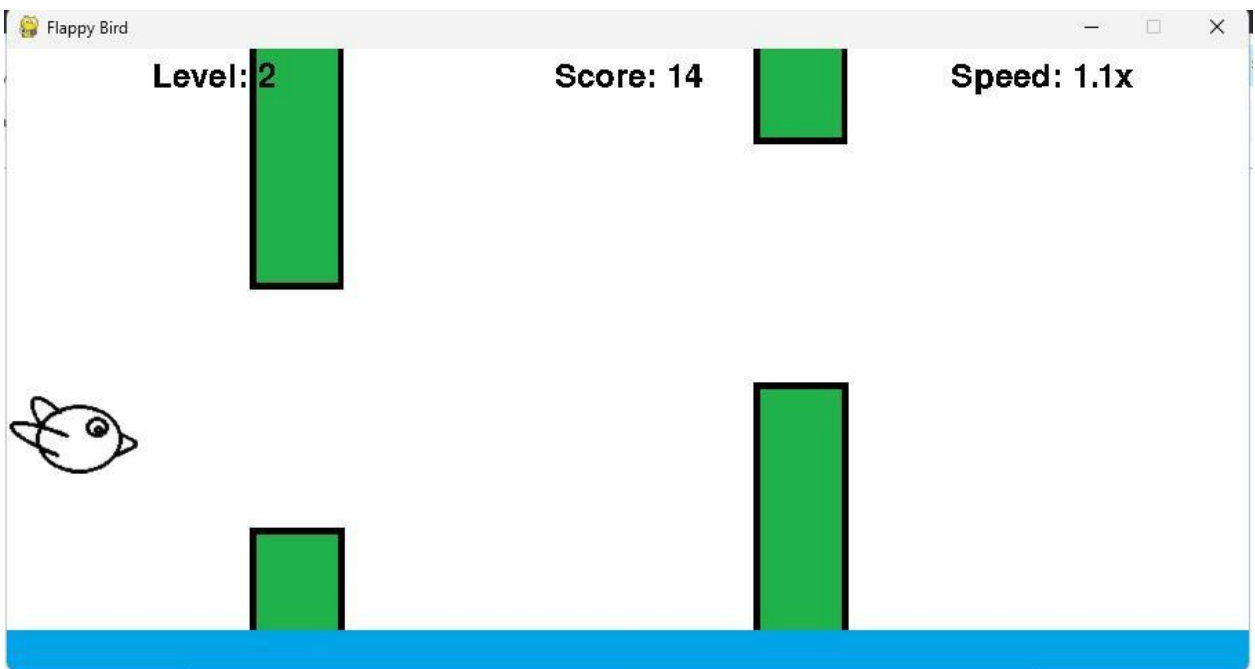


Press SPACEBAR or Click the button to Start
After Starting, use SPACEBAR or CLICK to flap!

Start



Game Active Interface



Game Retry Interface



Game Over! Your score: 17 Highscore: 17

Press SPACEBAR or click the button to retry

Player details and the highscore are updated to the database

Retry

Program Code

#This Function that checks for input validation on name,email and age_input

```
def input_validation():
```

```
while True:
```

```
    name = input("Enter your name: ")
```

```
        name.strip()
```

```
    if len(name)> 20 :
```

```
        print("Name must be less than or equal 20 characters")
```

```
    continue
```

```
    if name == "":
```

```
        print("Name cannot be blank")
```

```
        continue
```

```
    email = input("Enter your email: ")
```

```
    email.strip()
```

```

        if "@" not in email or "." not in email.split("@")[-1]:
            print("Please
enter a valid email address")
            continue
        else:
            parts = email.split("@")
            local_part = parts[0]
            domain = parts[1]
            if "." not in domain or
domain.startswith(".") or domain.endswith("."):
                print("Please enter
a valid email address")
                continue
            if not local_part or " " in local_part:
                print("Please enter a valid email address")
                continue
            for checkempty in
domain.split("."):
                if checkempty == "":
                    print("Please enter valid email address")
                    continue
            age_input = input("Enter your age: ")
            age_input.strip()
            if not age_input.isdigit():
                print("Age must be a valid number")
                continue
            age = int(age_input)
            if age < 8 or age > 120:
                print("Age must be at greater than or equal to 8 but less than or equal to 120")
                continue
            print("Validation
successful launching game....")
            return
name, age, email

#Function that sorts the score
def Partial_BubbleSort(scores_array):

```

```

swapped = True          n = len(scores_array)          while swapped:

    swapped = False          for i in range(n-1):
        if scores_array[i] < scores_array[i+1]:
            scores_array[i],scores_array[i+1] = scores_array [i+1],scores_array[i]

    return scores_array name,age,email = input_validation()

import pygame          import
mysql.connector
import random from sys import exit #Allows the program to exit properly

#This function establishes a connection to the local database server and creates a
database if it doesn't exist          def create_connection():          global
mydatabase,mycursor

    mydatabase = mysql.connector.connect(
        host="localhost",
user="root",
        password=""
    )

    mycursor = mydatabase.cursor()
mycursor.execute("CREATE DATABASE IF NOT EXISTS Playerdatabase")
mydatabase.database = "Playerdatabase"
mycursor.execute("""
    CREATE TABLE IF NOT EXISTS playerdetails (
        player_id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(255),
            email VARCHAR(255),
age INT,

```

```

        highscore INT,
        level INT,
        retries INT
    )
    """
create_connection()

#This function Updates or inserts player data everytime the player dies
def update_or_inserts_records(name, age, email,highscore,level,retries):    query =
"SELECT player_id,highscore FROM playerdetails WHERE name = %s        AND
age = %s AND email = %s"
mycursor.execute(query,(name,age,email))        result = mycursor.fetchone()

    if result:

player_id = result[0]
previous_highscore = result[1]                                if
highscore > previous_highscore:

        update_query = "UPDATE playerdetails SET highscore = %s,level =
%s,retries = %s WHERE player_id = %s"
        mycursor.execute(update_query,(highscore,level,retries,player_id))
    else:

        insert_query = "INSERT INTO
        playerdetails(name,age,email,highscore,level,retries)
        VALUES(%s,%s,%s,%s,%s,%s)"

        mycursor.execute(insert_query,(name,age,email,highscore,level,retries))

mydatabase.commit()

```

"""The Bird class represents the player so it handles the appearance,movement and collision mechanics of the player.The bird or the player character can be controlled

using this class."""

class

Bird(pygame.sprite.Sprite):

def __init__(self): #Constructor for Bird class

super().__init__()

self.bird_frame_1 = pygame.image.load('Bird/Bird_frame_1.png').convert_alpha()

self.bird_frame_2 = pygame.image.load('Bird/Bird_frame_2.png').convert_alpha()

self.image = self.bird_frame_2

self.rect = self.image.get_rect(midbottom=(50, 100))

self.gravity = 0

"""This function implements gravity on the bird and checks for collision with the

ground if a collision is detected true is returned else false"""

def handleGravityAndCollision(self): if game_active:

self.gravity += 0.35

self.rect.y += self.gravity

if self.rect.bottom >= 420:

self.rect.bottom = 420

return True

return False

def flap(self): #This function contains code to allow the bird to flap

if game_active:

self.gravity = -5.75

self.image = self.bird_frame_1

def stop_flap(self): #This function contains code to reset flap image

self.image = self.bird_frame_2

"""Pole class is responsible for npc (non-player controlled) characters which are the poles that the bird has to escape in order to survive. This class is responsible for movement, collision and appearance"""

class

Pole(pygame.sprite.Sprite):

def __init__(self, x, y, is_top): #Constructor for Pole class

```

    super().__init__()

    self.image = pygame.image.load('Vertical pole/Pole_Bottom.png').convert_alpha()

    #Sets image for bottom pole

    if is_top:

        self.image = pygame.image.load('Vertical pole/Pole_Top.png').convert_alpha()

        self.rect = self.image.get_rect(center=(x, y))
self.speed = 5

#This function updates the position of the pole and removes it from the game if it moves
off-screen.
    def update(self):
        if game_active:

            #As the game progresses the speed multiplier and speed is increased

            self.rect.x -= self.speed * speed_multiplier

            if self.rect.right < 0:

                self.kill() #Pole is removed if it goes out of the screen

pygame.init()
screen = pygame.display.set_mode((900, 450)) #900 x 450 resolution
pygame.display.set_caption('Flappy Bird')
clock = pygame.time.Clock()
water_surf = pygame.image.load('Water/water.png').convert_alpha() #Water image
#The single bird group is used for player character
bird = pygame.sprite.GroupSingle()
bird.add(Bird())

#This list contains pole objects to spawn infinite instances.Because using a "Group"
wouldn't meet project requirements.
obstacle_list = []
scores = [] #The scores array captures the player's score upon death

GAP_SIZE = 600
POLE_X_START = 950
POLE_Y_MIN = 75
POLE_Y_MAX = 300
#Game state
game_active = False #This variable defines the game active state
game_over = False #This variable defines the game over state
score_reset_time = 0 #

```

```

Tracks the start time of the score calculation          level = 1 #This variable keeps
track of player level and updates it                   retries = 0 #This variable counts the
number of retries the player makes    speed_multiplier = 1 #This variable keeps track
of the pole speed                                     next_level_score = 10 #This
variable keeps count of the next score needed to increase
the level and speed_multiplier
score_recorded = False #This variable ensures that score is recorded once
sorted = False #This variable ensures score is sorted once      highscore = 0
#This variable stores the highscore

#This function creates top and bottom poles with a random gap, adjusts their positions,
and adds them to the obstacle list.                      def
create_poles():
    gap_y = random.randint(POLE_Y_MIN, POLE_Y_MAX)          top_pole =
Pole(POLE_X_START, gap_y - GAP_SIZE / 2 - 330, True)        bottom_pole =
Pole(POLE_X_START, gap_y + GAP_SIZE / 2, False)
obstacle_list.append(top_pole)
obstacle_list.append(bottom_pole)

#This function dynamically updates the scores with intended logic and displays it
def update_and_display_score():
    """pygame.time.get_ticks gives the time since the game started in ms so the score is reset
    using score_reset_time, set when the game starts"""          score =
int(pygame.time.get_ticks() / 1000) - score_reset_time        score_surf =
font.render(f'Score: {score}', False, 'Black')                score_rect =
score_surf.get_rect(center=(450, 20))
    screen.blit(score_surf, score_rect)
return score

#This function displays the level and speed_multiplier during the game
def display_level_and_speed():
    level_surf = font.render(f'Level: {level}', False, 'Black')    level_rect =
level_surf.get_rect(center=(150, 20))

```



```

        screen.blit(level_surf, level_rect)
        speed_surf = font.render(f'Speed:
{speed_multiplier:.1f}x', False, 'Black')
        speed_rect =
speed_surf.get_rect(center=(750, 20))screen.blit(speed_surf, speed_rect)
font_large = pygame.font.Font(None, 50)
font = pygame.font.Font(None,
35)

# Creation of the start button

start_button_rect = pygame.Rect(350, 200, 200, 80)
button_text = font_large.render("Start", False, (0, 0, 0))

# Creation of the retry button

retry_button_rect = pygame.Rect(390,300, 120, 60)
button2_text = font_large.render("Retry", False, (0, 0, 0))

# Main loop
while True:
    screen.fill('White')
    bird.draw(screen)
    for event in pygame.event.get(): #loops through all events in pygame
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
            if
not game_active and not game_over:
            if
event.type == pygame.MOUSEBUTTONDOWN:
                if start_button_rect.collidepoint(event.pos):
                    game_active = True

                    #score_reset_time resets the score at the start of the game
                    score_reset_time = int(pygame.time.get_ticks() / 1000)

#The timer below is for the pole obstacle which is defined later
#changing 1200 below will change the poles spawn rate 1200 is in ms
pygame.time.set_timer(pygame.USEREVENT + 1, 1200)
                    if event.type ==
pygame.KEYDOWN:
                        if event.key == pygame.K_SPACE:
                            game_active = True
                            # Start the score timer

```

```
score_reset_time = int(pygame.time.get_ticks() / 1000)
pygame.time.set_timer(pygame.USEREVENT + 1, 1200)
```

```
#The bird can be flapped by either spacebar or mousebutton press if
game_active and event.type == pygame.KEYDOWN: if event.key ==
pygame.K_SPACE: bird.sprite.flap()
```

```

    if game_active and event.type == pygame.KEYUP:
        if event.key == pygame.K_SPACE:
bird.sprite.stop_flap( )
    if game_active and event.type == pygame.MOUSEBUTTONDOWN:
        bird.sprite.flap()
    if game_active and event.type == pygame.MOUSEBUTTONUP:
        bird.sprite.stop_flap()
    if game_over:
        if event.type == pygame.MOUSEBUTTONDOWN:
            if retry_button_rect.collidepoint(event.pos):
                game_active = True
                game_over = False
                bird.sprite.rect.midbottom = (50, 100)
                bird.sprite.gravity = 0
                score_recorded = False
                obstacle_list.clear()

            #Score time is reset for the next instance of score to be recorded accurately
                score_reset_time = int(pygame.time.get_ticks() / 1000)
                pygame.time.set_timer(pygame.USEREVENT + 1, 1200)
                level = 1

                speed_multiplier = 1

                retries += 1

                #Next level score is reset to update speed and level accurately
                next_level_score = 10

```

```

        if event.type == pygame.KEYDOWN:

            if event.key == pygame.K_SPACE:

                game_active = True

                game_over = False

bird.sprite.rect.midbottom = (50, 100)

bird.sprite.gravity = 0

score_recorded = False

obstacle_list.clear()

                score_reset_time = int(pygame.time.get_ticks() / 1000)

                pygame.time.set_timer(pygame.USEREVENT + 1, 1200)

                retries += 1

                level = 1

                speed_multiplier = 1

                next_level_score = 10

            if game_active and event.type == pygame.USEREVENT + 1:

                create_poles() #To add poles when the game is active

            if not game_active and not game_over: #Display for game menu

                pygame.draw.rect(screen,(0,255,0), start_button_rect)

screen.blit(button_text, (start_button_rect.x + 50, start_button_rect.y + 20))

start_text = font.render("Press SPACEBAR or Click the button to Start",False-

-, 'Black')

                start_rect = start_text.get_rect(center=(450, 150))

screen.blit(start_text, start_rect)

            if game_active:

                """The for loop updates each pole, draws it on screen, and ends the game if

```

```

        collision with the bird occurs."""
        pole.update()
        pole.rect)
        if pygame.sprite.collide_rect(bird.sprite, pole): #Collision Mechanics
            game_active = False
            game_over = True
        break
        score =
    update_and_display_score()

    if score >= next_level_score:
        level += 1
    next_level_score += 10
    speed_multilpier += 0.1
    if speed_multiplier < 2 :

    display_level_and_speed()

    #handleGravityAndCollision method takes care of bird gravity and returns true if
    the bird collides with the ground
    if bird.sprite.handleGravityAndCollision():
        game_active = False
        game_over = True

    if game_over:
        #score should be recorded only once and to do that score_recorded exists
        if not score_recorded:
            scores.append(score)

        score_recorded = True

        #score should be sorted only once and to do that score_recorded exists
        if not sorted :
            scores = Partial_BubbleSort(scores)

```

```

    if scores:
        highscore = scores[0]

        #Next few lines of code is focused on the UI of the game
        pygame.draw.rect(screen, (0, 255, 0), retry_button_rect)
        screen.blit(button2_text, (retry_button_rect.x + 15, retry_button_rect.y + 15))
        game_over_text = font.render("Game Over! Your score: " + str(score) +
        "Highscore: " + str(highscore) + ", True, 'Black')
        game_over_rect =
        game_over_text.get_rect(center=(450, 150))
        screen.blit(game_over_text, game_over_rect)

        retry_info_text = font.render("Press SPACEBAR or click the button to retry", True,
        'Black')

        retry_info_rect = retry_info_text.get_rect(center=(450, 200))
        screen.blit(retry_info_text, retry_info_rect)

        """The method below updates the records in the database or inserts the data like
        highscore,level and retries"""
        update_or_inserts_records(name, age, email,highscore,level,retries)

        #The next few lines of code is focused on UI again
        database_updatation_text = font.render("Player details and the highscore are
        updated to the database", True, 'Black')

        database_updatation_rect = database_updatation_text.get_rect(center=(450, 250))
        screen.blit(database_updatation_text,database_updatation_rect)

        screen.blit(water_surf, (0, 420)) #water image

        pygame.display.update()
        clock.tick(60)

```

SQL Code

The Queries mentioned are implemented in the program code above

Query 1:

```
CREATE DATABASE IF NOT EXISTS Playerdatabase
```

Query 2:

```
CREATE TABLE IF NOT EXISTS playerdetails (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    age INT,  
    email VARCHAR(255),  
    highscore INT,  
    level INT,  
    retries INT  
)
```

Query 3:

```
SELECT player_id,highscore  
FROM playerdetails WHERE name  
= {name input} AND age = {age input} AND email = {email input}
```

Query 4a:

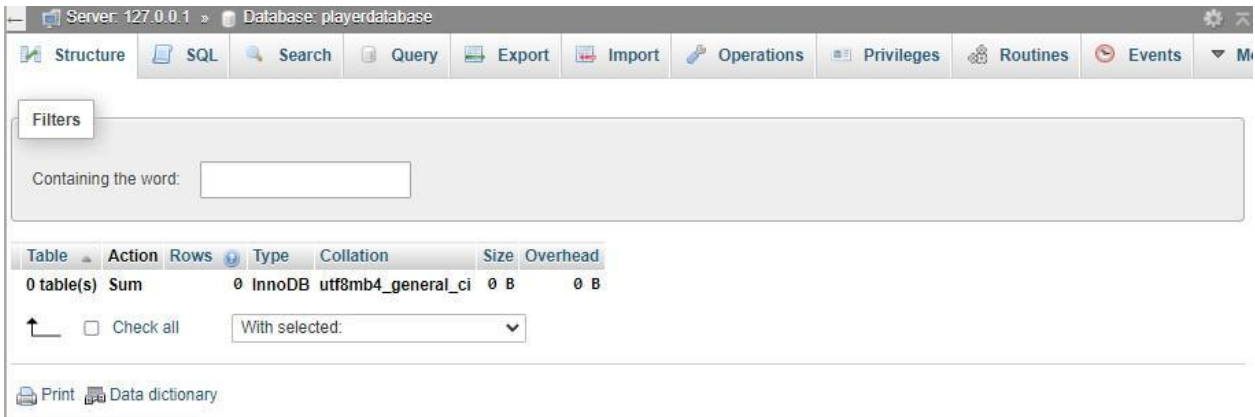
```
UPDATE playerdetails  
SET highscore = {highscore variable}, level = {level variable}, retries = {retries variable}  
WHERE player_id = {player_id variable} #player_id is the player id of the above query
```

Query 4b:

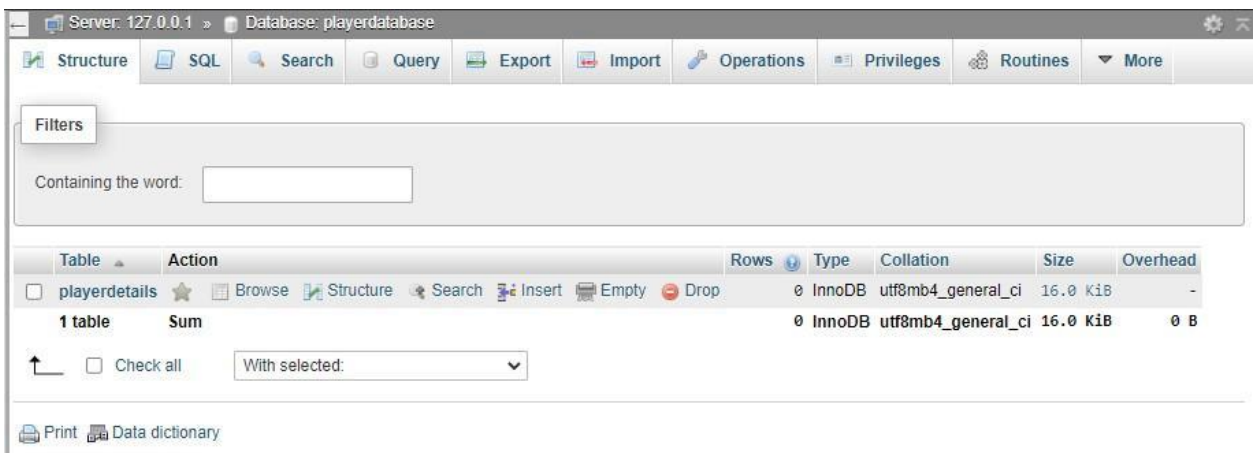
```
INSERT INTO playerdetails(name,age,email,highscore,level,retries)  
VALUES ({name input}, {age input}, {email input}, {highscore variable}, {level variable},  
{retries variable})
```

Screenshots of Database

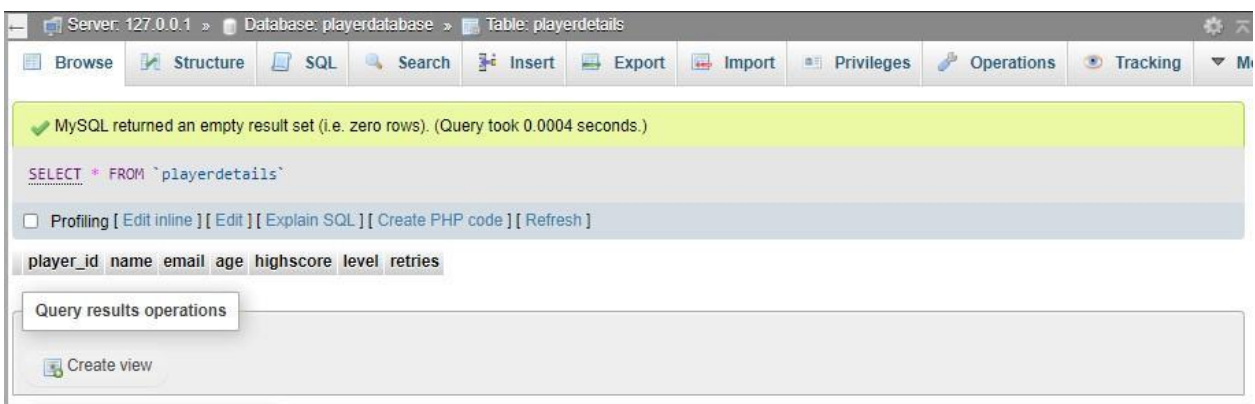
Before Starting game (during input validation) Database is created but no tables exists



After Input Validation a table is created, Records are updated/inserted when the player dies.



If the game is closed directly without recording any scores there is no data because all fields are required to register a record.



Record data is shown below with all data registered

SELECT * FROM `playerdetails`

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	player_id	name	email	age	highscore	level	retries
☐ Edit Copy Delete	1	Test Case Working	TestCaseWorking@gmail.com	17	17	2	2
☐ Edit Copy Delete	2	Namish Sharma	namishsharmayupme@gmail.com	65	11	2	0
☐ Edit Copy Delete	3	Test	Testlol@gmail.com	38	21	3	2
☐ Edit Copy Delete	4	Test2	testlol@gmial.com	34	6	1	0
☐ Edit Copy Delete	5	Namish	sharamanamish6@gmail.com	23	10	2	0

Research and development of new skills and knowledge

To create my flappy bird game I researched skills and implemented it in the game. These skills included

- 1) Learning a whole new library, Pygame which is an open source library for writing video games. I learnt a lot of details about video games in the process of the backend, frontend and overall game design. Most of the in-game mechanisms like interaction with the ground, enemies in general were done using rectangles rather than storing each matching pixels as a separate entity. Rectangles are placed on images which makes it easier to detect collisions, switch images and also interact with surfaces i.e. water.
- 2) Learning movement mechanisms is also a core skill in a particular game, for example when jumping the player character the keyboard/mouse inputs are taken . I understood how games are rendered in real time which was a fascinating concept which includes rendering multiple images and working accordingly with keyboard or mouse input. I learnt to use various built in modules of pygame such as creating a timer, creating buttons and score recorder mechanism.
- 3) I learnt how to create an UI in pygame for my game as shown in the screenshots. Various concepts such as 2d plane coordinates and basic shape properties such as colour, anti aliasing and size can be modified using pygame itself. I learnt quite a few image handling methods such as rotozoom and size. Pygame also involves displaying text in any downloadable font and modifying its attributes such as text colour and size etc.
- 4) Learning pygame allowed me to have more practical experience with object oriented programming and develop on my core essentials. I watched this youtube tutorial "Ultimate introduction to pygame" which was a good starter for the core essentials.

The website I then was pygame.org and w3schools to elaborate on the basic concepts.

5) The functional requirements that required the new skills is listed below :

1.The game must allow the bird to fall naturally due to gravity unless the player presses the mouse button or the spacebar to make it ascend.

2.Display the bird, poles, and ground.

3.Animate the bird's flapping by switching images when flapping and flying.

4.The game will provide a start and game-over screen.

5.The game must generate poles with a fixed gap at regular intervals.

6.The game must display the end game screen after collision is detected and interactive buttons such as “Start” and “Retry”.

7.The game must detect user inputs like spacebar, collisions and button press

8. The game asks for player’s details such as player name,email,age. The game records high scores,retries and level for each individual player.

10.The current score will be displayed during the gameplay, along with the level and speed of the obstacles thus allowing players to track their progress.

11.The game tracks the user's score everytime the user dies the score is stored in the scores array.

13.The game will allow players to retry when they collide with the vertical poles or touch the ground in-game.

14.As the game progresses the speed of the vertical poles is increased

15.The game must display a start screen with a play button.

Log of ongoing testing

Functional Requirement(s) being tested	Describe any issues encountered	Resolution of issue	Any references used (if required)
2.Display the bird, poles, and ground.	The images were not clear and blurry when movement occurred .	Convert alpha which changes the pixels of an image to include the alpha slot in the rgb colour values	https://www.pygame.org/docs/ref/surface.html#pygame.Surface.convert_alpha

9.The game will validate inputs which are the	The player email validation wasn't rigid enough.	I had to check for a lot of test cases such as inputting no domain or local name. This system	
---	--	---	--

player name,email and their age.		is currently quite rigid. I haven't found a drawback and it's fully tested.	
1.The game must allow the bird to fall naturally due to gravity unless the player presses the mouse button or the spacebar to make it ascend.	The bird had an option to skip all poles by flying higher than the poles. The gravity was also inconsistent as it slowed the bird down at a constant speed rather than accelerating downwards.	The issue was resolved by making the top pole bigger than the bottom pole The issue was fixed by increasing gravity at lower increments to give a proper acceleration.	https://www.pygame.org/docs/ref/image.html
18.A query is created to update the highscores of the player if his current highscore is greater than his previous highscore	Duplicate records were created when player had same name in the database	To resolve this issue I had to introduce two more functional requirements 18 and 19 which updates or adds the records depending whether the player existed	https://www.w3schools.com/python/python_mysql_getstarted.asp

13.The game will allow players to retry when they collide with the vertical poles or touch the ground in-game.	Retry did not remove the variable values for current score, speed multiplier, and level	everything had to be reset after the retry button was pressed.	
3.Animate the bird's flapping and falling.	The “rectangles” for each bird were not the same. Therefore incorrect collisions were registered.	Since pygame draws the rectangle of a class. I had to manually make the image aligned by keeping the same	https://www.pygame.org/docs/ref/rect.html
		size for both birds but ensuring consistency when looped.	
14.As the game progresses, the speed of the vertical poles shall increase up to a predefined maximum speed limit.	There was no cap on the speed for poles. So it would have become unplayable after a certain duration.	A cap at 2x is added so the game won't progress any further from the cap.	
7.The game constantly checks for collisions between player and the vertical poles during the game to display the game-over phase.	After game-over phase was initiated and retry button was pressed the bird still kept colliding with the poles causing the game to end always	The timer for poles and the bird location was reset therefore a new game loaded.	https://www.pygame.org/docs/ref/time.html

Testing of the Problem

Comprehensive test plan

Test Number	End-user/Functional Requirements to be tested	How it will be tested (including values if required)	Expected Results
Test 1	1.The game must allow the bird to fall naturally due to gravity unless the player presses the mouse button or the spacebar to make it ascend.	The game is launched and the bird's gravity is taken into consideration and tested to check if the bird falls at a constant rate.	The bird must accelerate downwards at a constant rate.
Test 2	2.Display the bird, poles, and ground(water).	Launch the game and check if all elements appear correctly on the screen.	The bird, poles, and ground should be visible without graphical glitches. The elements must also appear correctly in their correct position
Test 3	3.Animate the bird's flapping and falling.	Check the image switches by pressing spacebar or using mouse button	The images should be properly synchronized and depict a "flying bird,". It must function smoothly without any graphical glitches.

Test 4	4.The game will provide a main menu and a game-over screen.	Launch the game and check for both the game menu and death screen.	The game menu phase must be invoked first. The death screen must be displayed when the bird collides with the ground(water) or the poles.
Test 5	5.The game must generate poles with a fixed gap at regular intervals.	The pole generation must be verified for consistency, and the timer should function correctly for accurate testing.	The distance between poles must remain consistent, ensuring they are generated with the same gap while appearing at different positions.
Test 6	6.After a collision is detected the game must show the death screen with the “Retry” button.	Launch the game and check for the death screen after colliding with a pole.	The death screen must show the scores, highscore and must contain a retry button.
Test 7	7.The game constantly checks for collisions between player and the vertical poles during the	Check for the death register when the bird hits the pole or touches the ground or flies high enough and	The game must transition to the end game screen when the player touches the ground(water) or vertical
	game to display the game-over phase.	hits the pole.	poles.

Test 8	8. The game asks for player's details such as player name,email,age. The game records high scores,retries and level for each individual player.	Launch the game to check for inputs such as player name,email,age. The database is visited to check for accurate data capture for player details such as scores, level etc.	The game must display text boxes and instructions, such as a prompt to enter the player's name. The database must also have valid player details recorded.
Test 9	9.The game will validate keyboard inputs, which are the player's name, email, and age.	Test data is entered to check validation for each field. Show an example for accurate and incorrect test cases. Blank fields, invalid names, are tested for accurate data capture.	The game must accept valid data entries while rejecting invalid inputs with an appropriate error message.
Test 10	10.The current score will be displayed during the gameplay, along with the level and speed of the obstacles thus allowing players to track their progress.	Check for UI elements in the game active phase showing the progressive stats of the player and the impact of the stats on the bird.	The UI must show the score, high score, and speed multiplier during the game and ensure they update dynamically, remain visible without obstructing gameplay, and accurately reflect real-time changes in player progress.

Test 11	11.The game must track the user's score, and	The scores array is tracked by printing the value of score	The score must be added to the list, when the player dies. The
---------	--	--	--

	upon the player's death, store the score in the scores array.	during the game(in the game-active phase). The values during the game are compared with the expected values.	sorting algorithm is halted for now so the output will not have an sorted scores array.
Test 12	12.The game will implement a “partial” bubble sort algorithm on the scores array and the highest score is displayed using the “partially” sorted score’s array.(Partial bubblesort explained later in evaluation).	The scores array is tracked by printing the value of score during the game(in the game-active phase). The values during the game are compared with the expected values. The highscore is shown after the game ends.	The sorting algorithm is run alongside and the scores array will be sorted.
Test 13	13.The game will allow players to retry when they collide with the vertical poles or touch the ground in-game.	Retry button is tested and the game state is reverted back to the game-active phase.	The retry button must reset all current session data values and load a brand new game with the default spawn locations.

Test 14	14.As the game progresses, the speed of the vertical poles shall increase up to a predefined maximum speed limit.	The speed multiplier is checked for its functionality and impact on the bird.	The speed multiplier must increase to 2 for the first 10 levels and then levels off.
Test 15	15.The game must display the game menu	Launch the game and check for the game	The game menu must have instructions on how to control the bird

	with instructions and with the “Start” button.	menu and buttons.	and must contain a “Start” button.
Test 16	16. Connection to a database is achieved and a Player database is created.	The game shall automatically create a database upon execution.The database is hosted locally on your pc therefore the connection with the database is checked.	The database must have a player details table which contains no records. This shows that the connection with your program and database is established.
Test 17	17.A query is created to generate the playerdetails table.	The game shall automatically create a database upon execution.	The table must have no records upon execution.

Test 18	18.A SELECT query is created to find matching records of an existing player to modify his details for update query	The current player plays another game using the “retry” button.	The record of the player must be updated after the player dies proving the SELECT query returned a result and the details of the player must be updated. If the SELECT query does not return a result the player data is inserted.
Test 19	19.A UPDATE query is created to update the highscores of the player if his current highscore is greater than his previous highscore	The player plays another game with a highscore greater than the previous highscore and then the database record is then checked.	The player must have his highscore, level and retry updated which proves that the update query is executed.
Test 20	20.A INSERT query is created to insert the player details if player records do not exist.	A new game session is loaded and the game is played. When the player dies for the first time his records are inserted in the table.	The playerdetails table must contain a new record of the player details who played the new game session.
End-User requirements Testing			

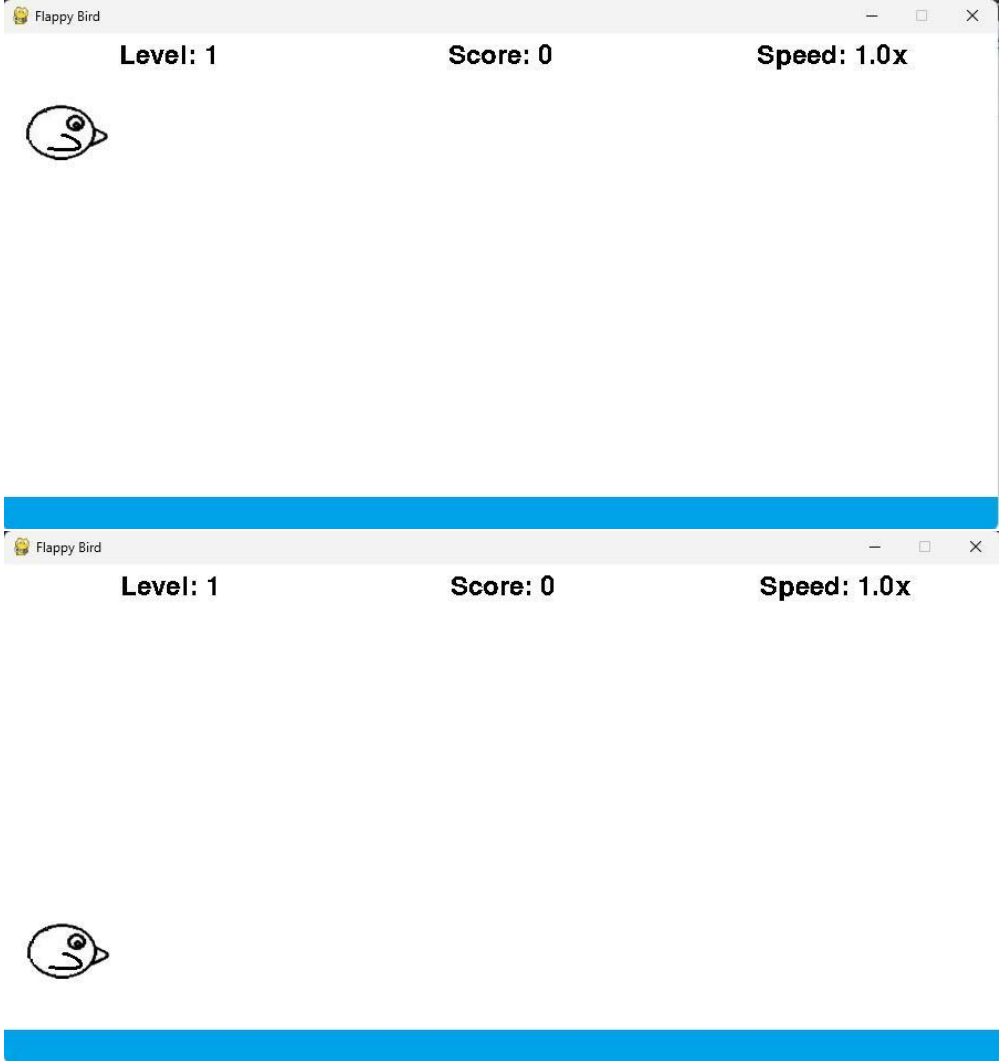
Test 21	1.Inputs that are taken must have input validation, should provide clear instructions, and display error messages for empty or incorrect fields	The input validation is checked again with different test values for different error messages.	The instructions for the inputs must be detailed. There must be clear feedback on errors when the player enters an invalid input.
Test 22	2.The user-interface must be clear enough to understand the controls and mechanics without needing detailed instructions.	Comment on the usability of the user interface and ease of use for the end user.	The user interface must be clear enough to understand the game mechanics without needing detailed instructions.
Test 23	3.The game should become progressively challenging over time.	The obstacle speed increases, and the game provides feedback on the difficulty.	The speed multiplier will increase to 2x if the player doesn't die and stays constant.
Test 24	4.Players should be able to see their current score and high score,	The score must be displayed when the player plays the game and both the	The game sends the scores to the database for each individual who runs a new game
	allowing them to track their progress and set goals for improvement.	score and highscore must be displayed after death.	session so the highscore for each player can also be viewed and future goals can be set.

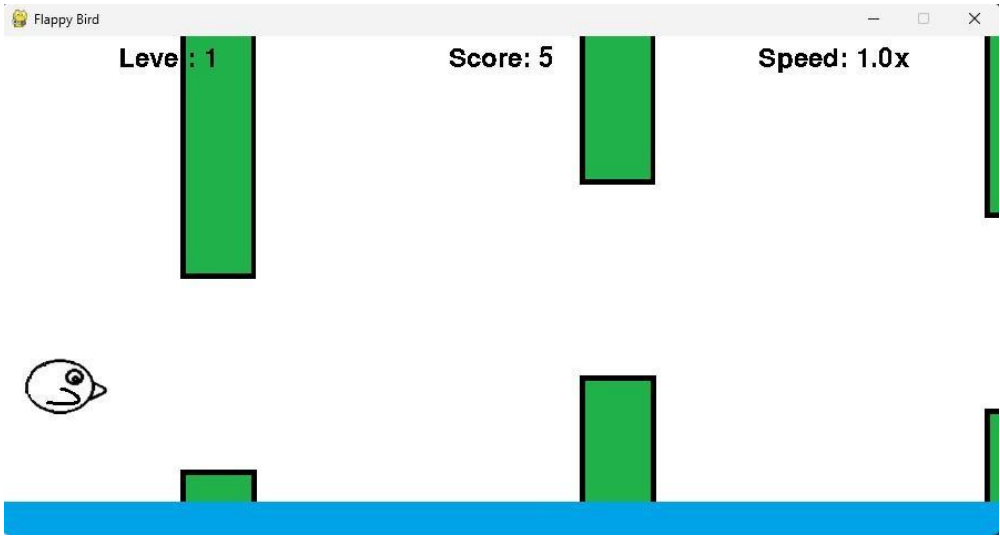
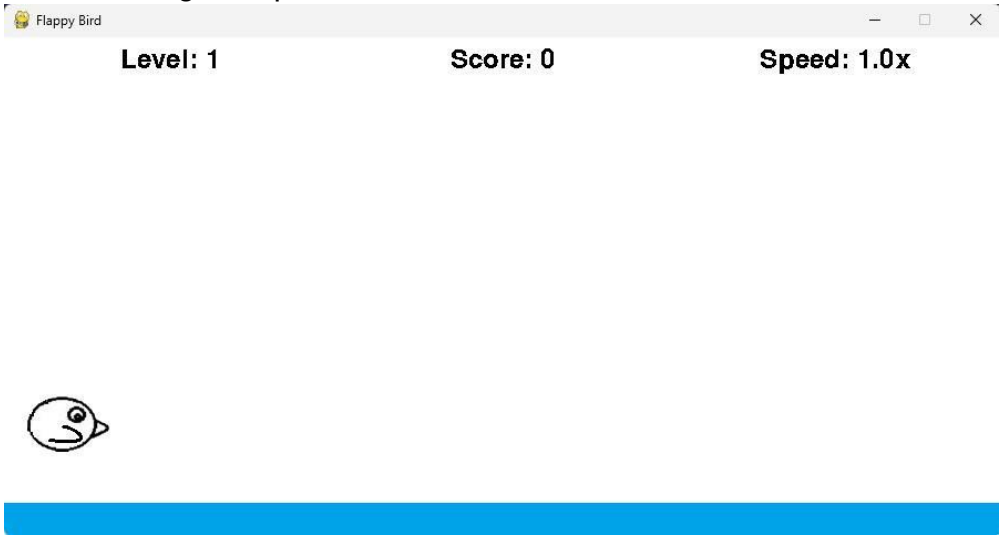
Test 25	5.Players should have the option to start the game when required.	The Start button is tested to check if the player can start the game at will.	This is an essential end-user requirement because it allows the player to initiate the game. Without it, the user may not have control over when the game begins.
---------	---	---	---

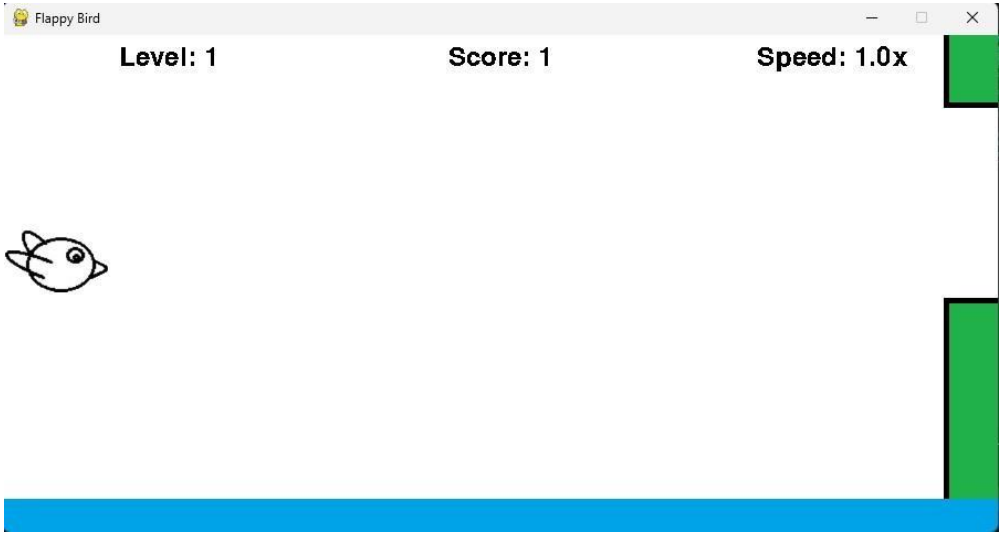
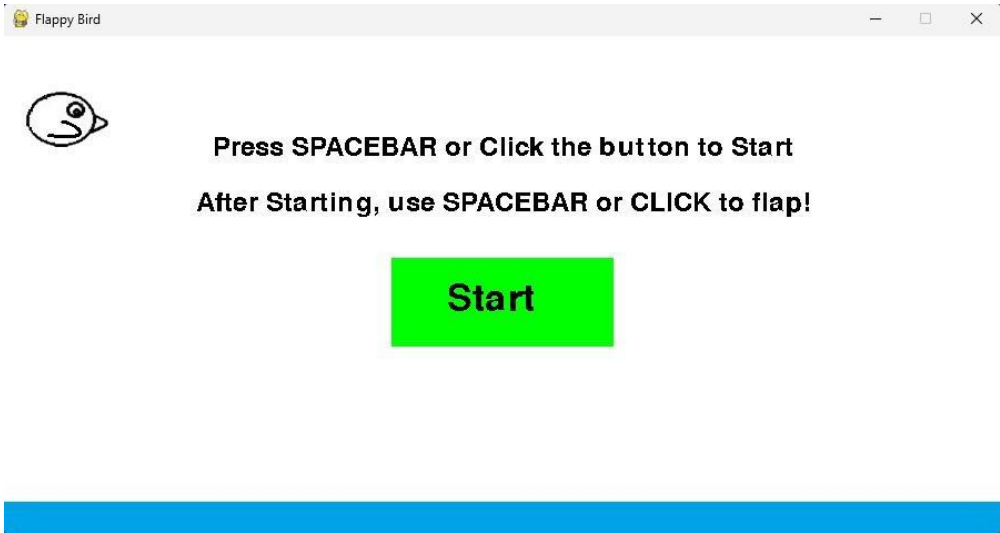
Persona and test cases

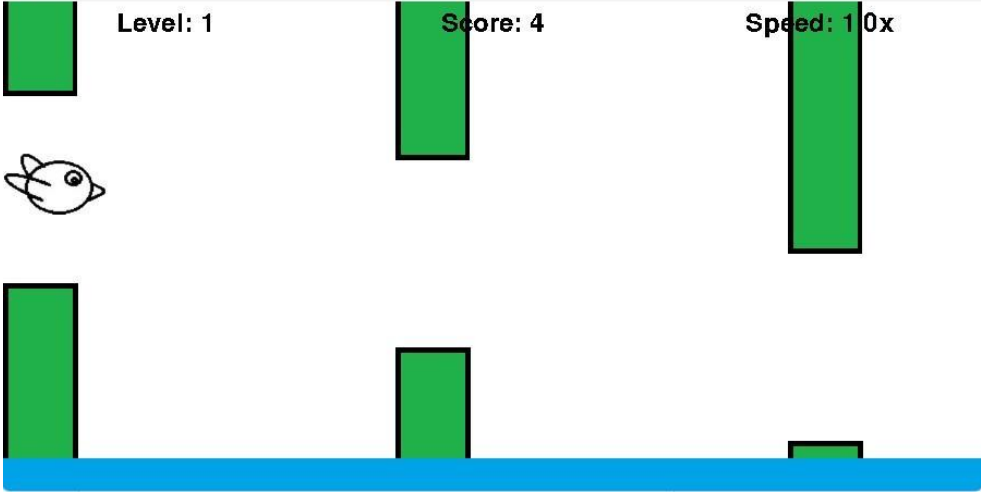
	Characteristics of Persona	Test case(s) for this persona
Persona	Bill is 13 years old and has never the iconic flappy bird game, but he has played other runner games such as the offline dino game in chrome. His friend challenges him to the game and they play different game sessions in the same PC. Since the data is recorded in the database, The overall statistics of the player is accessed using the database.	1.Launch the game. 2.Check if Bill has trouble in the input validation phase to input his data. 3.Check if Bill can read the instructions and error messages precisely and provide accurate data input. 4.Check if Bill can recognize the game menu screen, read the instructions and start playing the game. 5.Observe if Bill understands the game mechanics such as controlling the bird and avoiding the poles. 6.Check if Bill can recognize the game-over screen and restart the game. 7.Check if Bill can navigate to the database and compare his score with his friends who play different
		game sessions with him.

Evidence of requirement testing

Test Number	Testing evidence
Test 1	Gravity implementation  The evidence consists of two screenshots of the Flappy Bird game interface. Both screenshots show the title bar 'Flappy Bird' with standard window controls. The game status at the top of each screen is 'Level: 1', 'Score: 0', and 'Speed: 1.0x'. In the top screenshot, the bird is positioned near the top of the play area, and a solid blue horizontal bar representing the ground is at the bottom. In the bottom screenshot, the bird is positioned near the bottom of the play area, just above the same solid blue ground bar. This visual comparison demonstrates the effect of gravity on the bird's movement.
Test 2	The bird, poles, and ground should be fully rendered, in their correct positions, and without overlapping or missing textures.

	 A screenshot of the Flappy Bird game window. The title bar says "Flappy Bird". The game area shows "Level: 1", "Score: 5", and "Speed: 1.0x". A small bird is in flight, and there are green pipes and a blue ground.
Test 3	Before clicking the spacebar.  A screenshot of the Flappy Bird game window. The title bar says "Flappy Bird". The game area shows "Level: 1", "Score: 0", and "Speed: 1.0x". The bird is on the ground, and there is a blue ground. After clicking the spacebar.

	 A screenshot of the Flappy Bird game window. The title bar says "Flappy Bird". At the top, it displays "Level: 1", "Score: 1", and "Speed: 1.0x". The game area shows a yellow bird on the left, a blue ground bar at the bottom, and two green pipes on the right. The window has standard minimize, maximize, and close buttons.
Test 4	Start screen  A screenshot of the Flappy Bird start screen. The title bar says "Flappy Bird". On the left is a yellow bird icon. In the center, the text reads: "Press SPACEBAR or Click the button to Start" and "After Starting, use SPACEBAR or CLICK to flap!". Below the text is a large red button with the word "Start" in black. At the bottom is a blue ground bar. Death screen

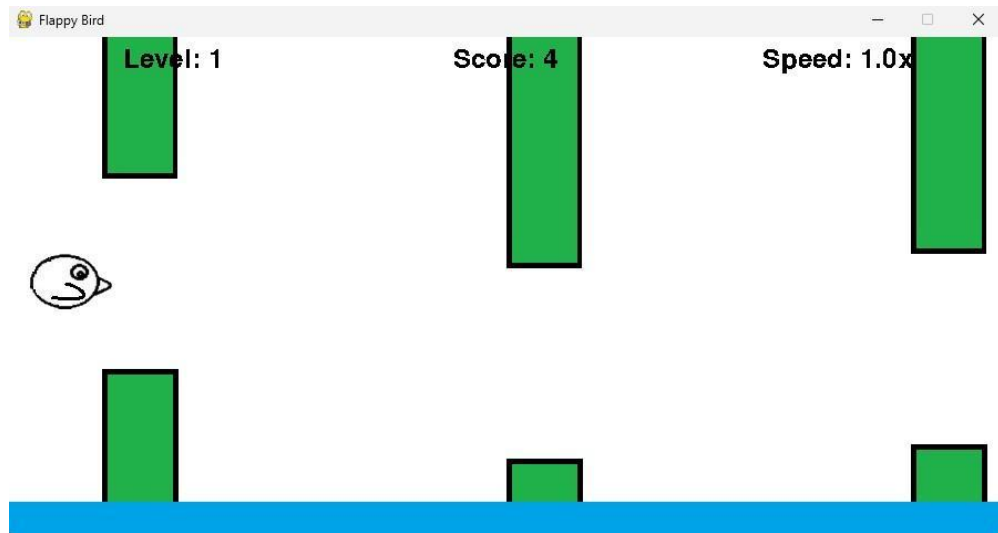
	Flappy Bird Game Over! Your score: 2 Highscore: 2 Press SPACEBAR or click the button to retry Player details and the highscore are updated to the database Retry 
Test 5	Pole generation evidence Flappy Bird Level: 1 Score: 4 Speed: 10x 
Test 6	The image shows the UI elements such as highscore, score and retry button shown correctly in the death screen.

	Flappy Bird Game Over! Your score: 2 Highscore: 2 Press SPACEBAR or click the button to retry Player details and the highscore are updated to the database Retry 
Test 7	Game active phase (Collision not detected) Flappy Bird Level: 1 Score: 1 Speed: 1.0x  Game-over Phase(Ground collision detected)

	Flappy Bird Game Over! Your score: 2 Highscore: 2 Press SPACEBAR or click the button to retry Player details and the highscore are updated to the database Retry 
Test 8	Player input phase PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + ^ ⌵

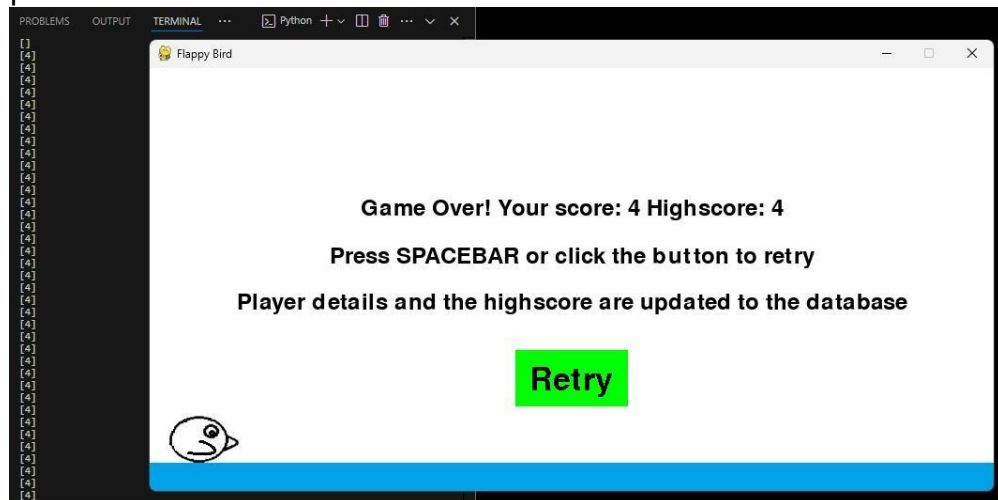
Test 10

The image contains the visual elements such as score and score_multiplier in the game active phase.



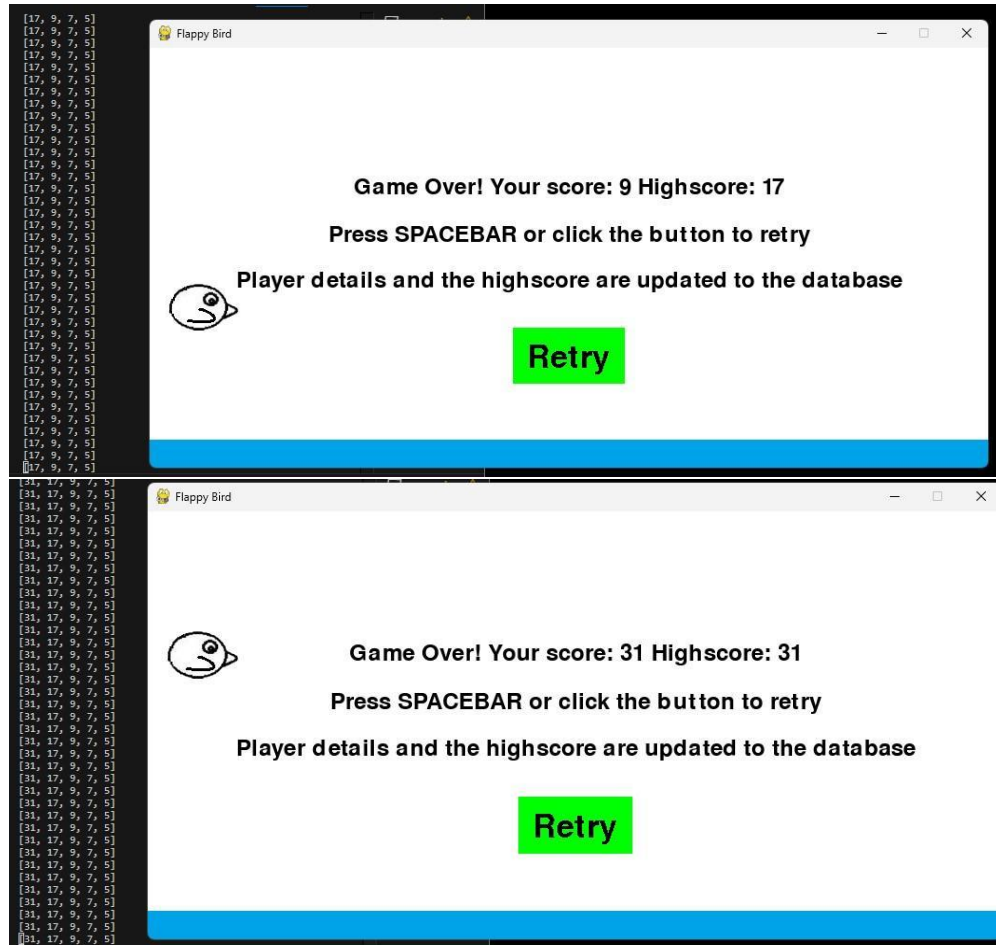
Test 11

The values of the score array is shown in the screenshot below. Initially the array was empty, but when the player died the array was updated.



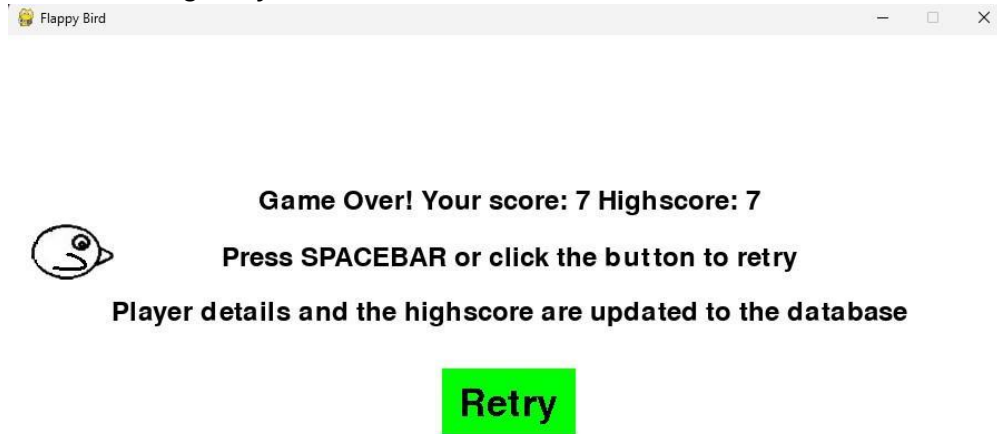
Test 12

The pictures below show a test run of the game while inspecting the scores array. In the first picture, the scores array contains [17, 9, 7, 5]. In the second picture, the array is updated to [31, 17, 9, 7, 5] after the player's death. The high score is also displayed on the screen after sorting and retrieving the highest value.

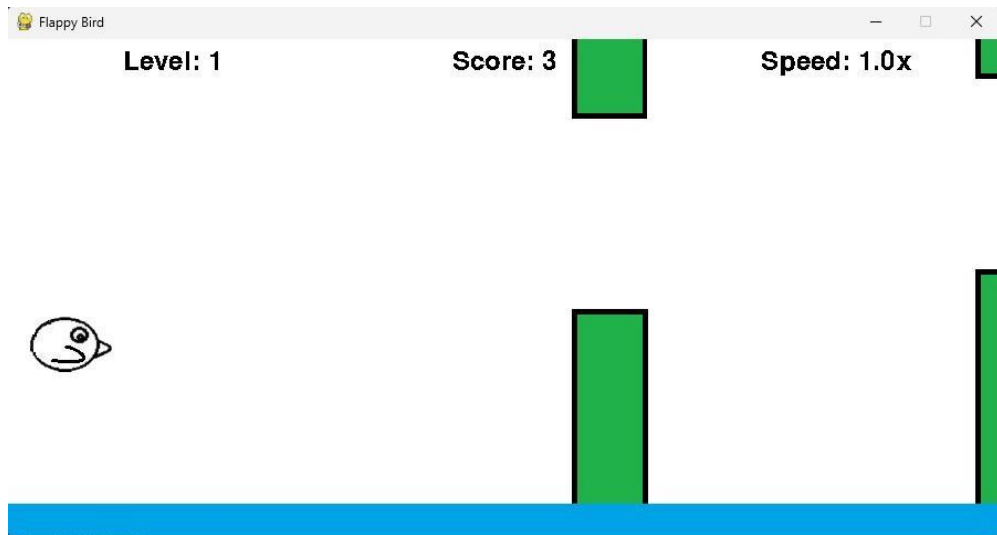


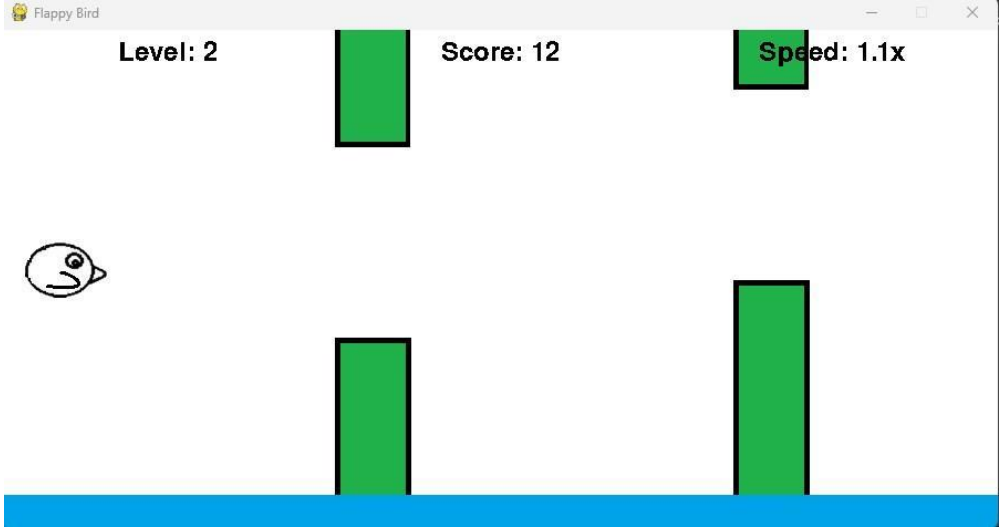
Test 13

Before clicking retry button



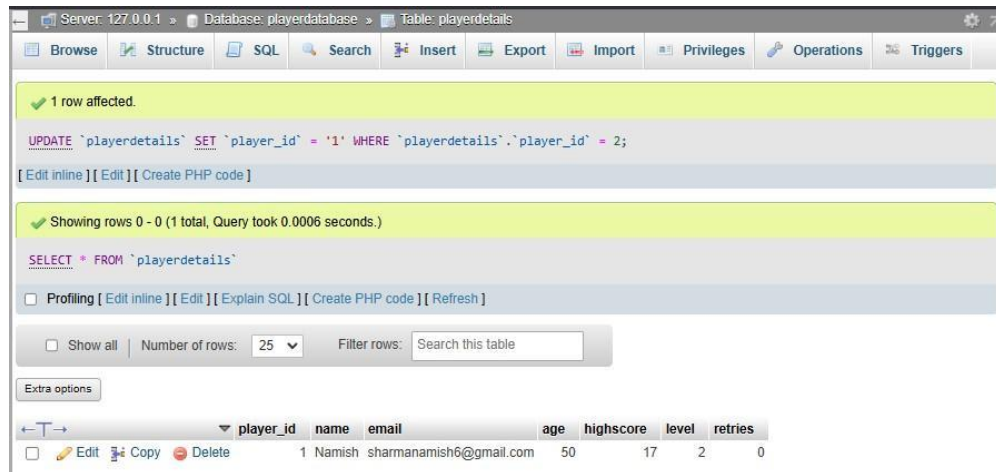
After clicking retry button (game successfully rebooted)



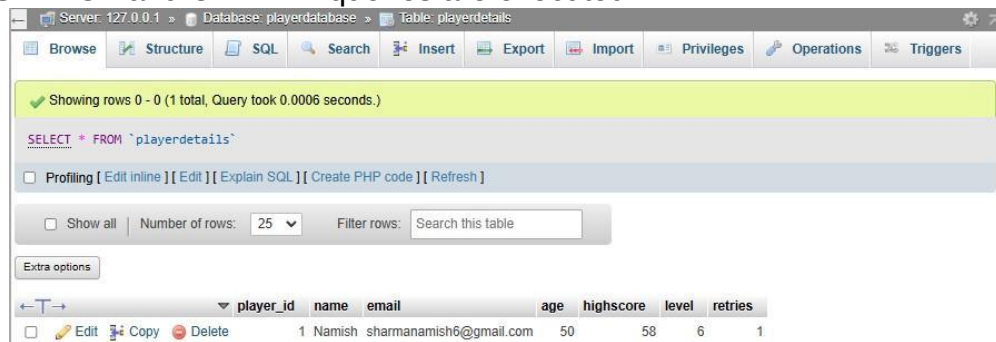
Test 14	The bird's speed does not actually increase with the speed multiplier. Instead, the poles move faster, creating the illusion that the bird is flying forward. Throughout the game, the bird remains in the same position, only flapping its wings while the poles move past it. 
Test 15	The game menu shows the start button and instructions needed for a beginner to understand the layout of the game.



Test 16	The screenshot below shows the Player database created after program execution. 
Test 17	The screenshot below shows the playerdetails database which is created after program execution. 
Test 18	A game is played, the player's details are inserted into the table, confirming that both the SELECT and INSERT queries have been executed.

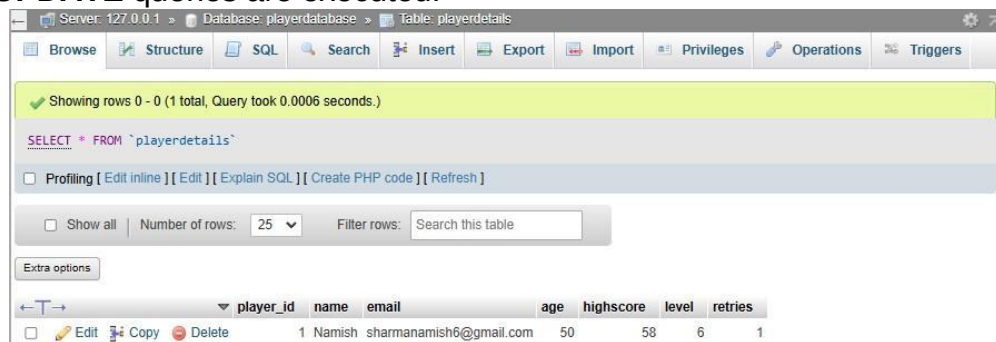


In the same game session, another game is played using the retry button. If the player's new high score exceeds the previous high score, the **SELECT** and **UPDATE** queries are executed.



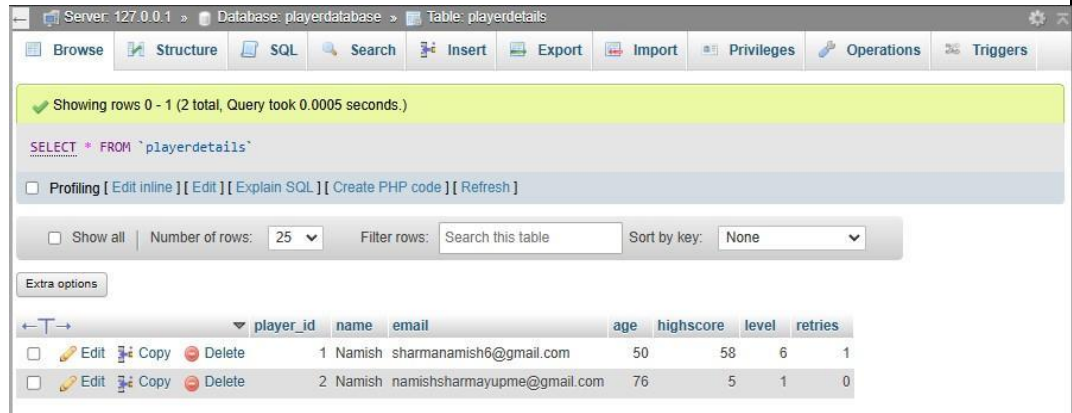
Test 19

The player in the previous picture surpasses the high score by playing another game session using the Retry button. As a result, the **SELECT** and **UPDATE** queries are executed.



Test 20

The game is quit, and a new game session is started with different player details. When the player dies, the INSERT query is executed since the SELECT query will not return any results. The second record below shows the new game session.

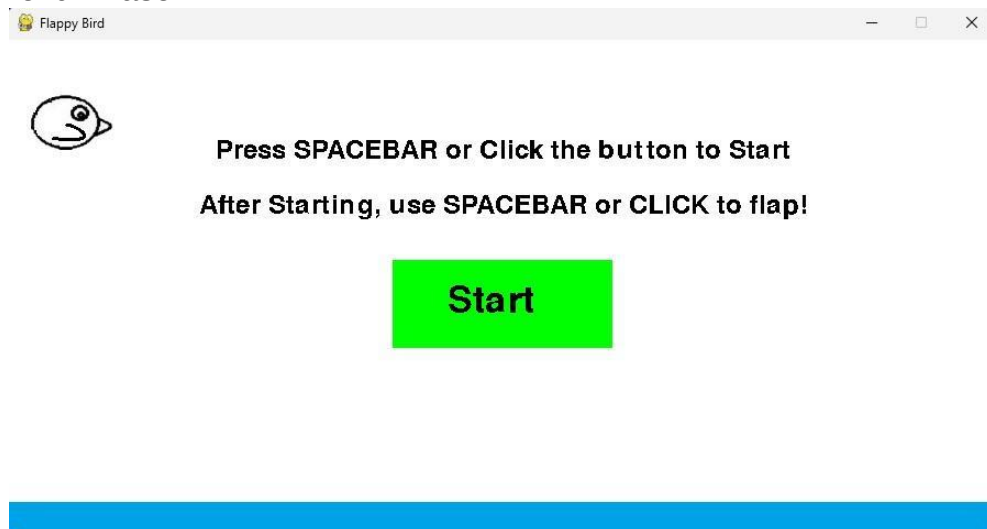


player_id	name	email	age	highscore	level	retries
1	Namish	sharmanamish6@gmail.com	50	58	6	1
2	Namish	namishsharmayupme@gmail.com	76	5	1	0

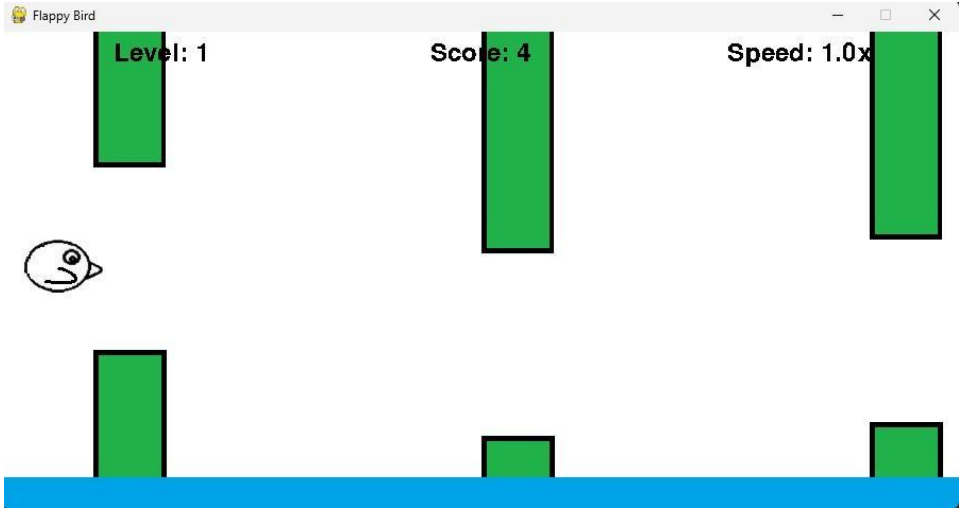
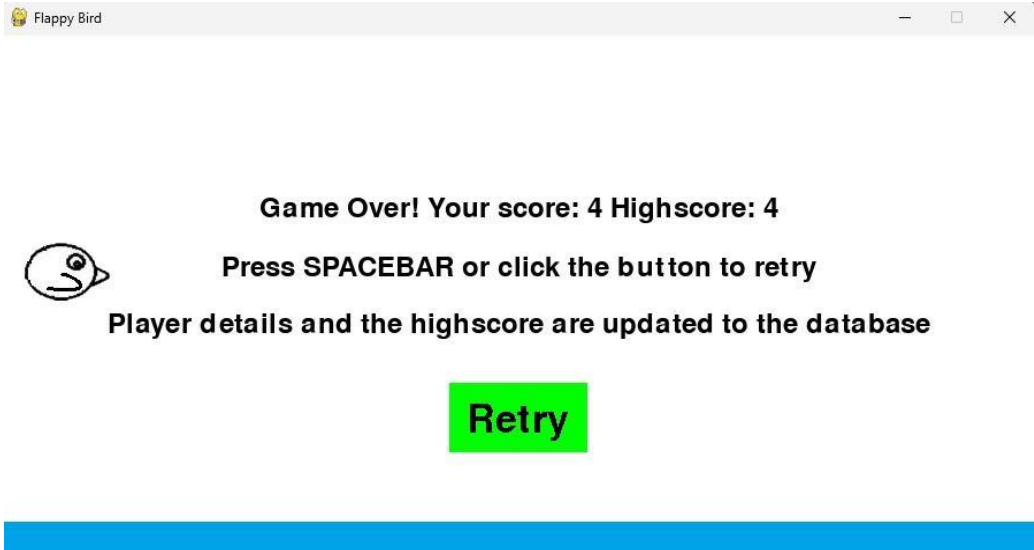
Test 21

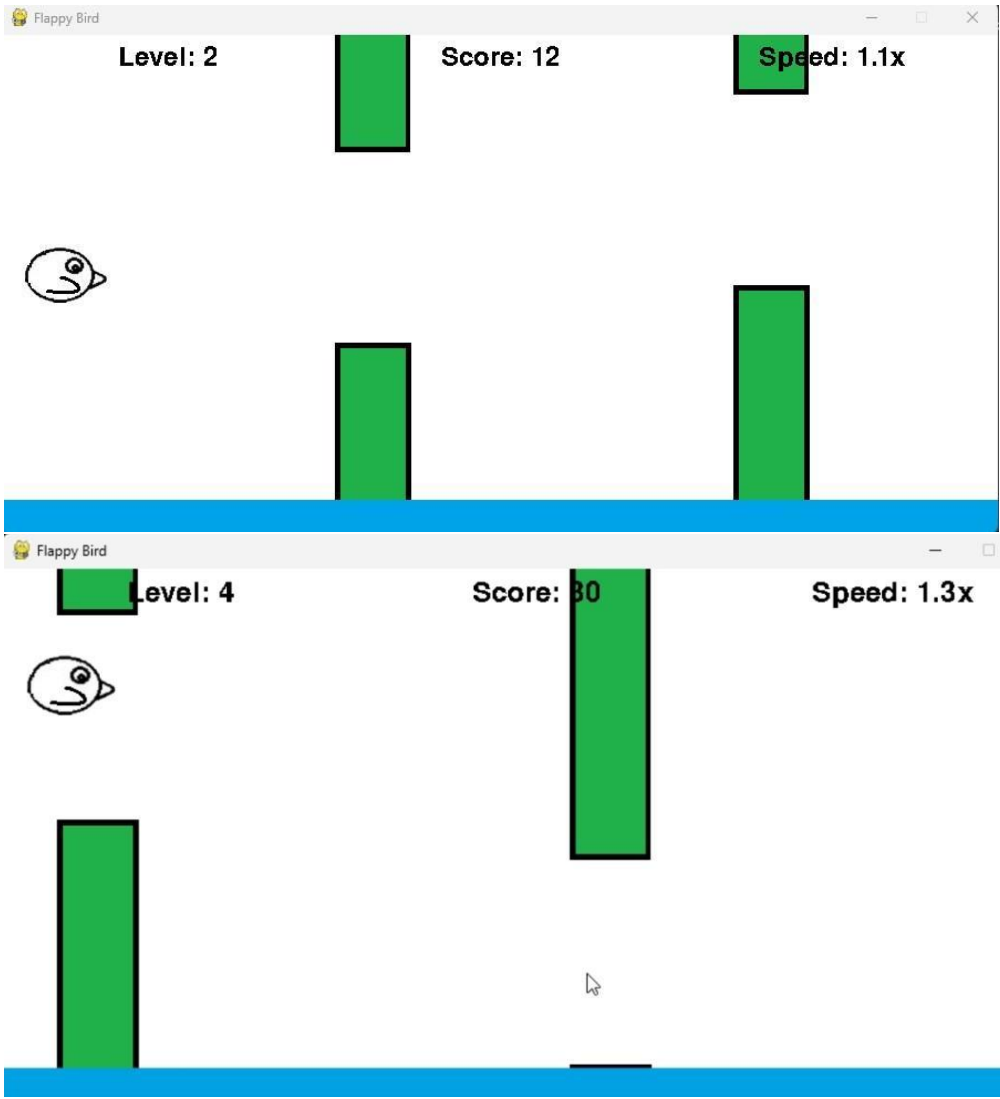
Test 22

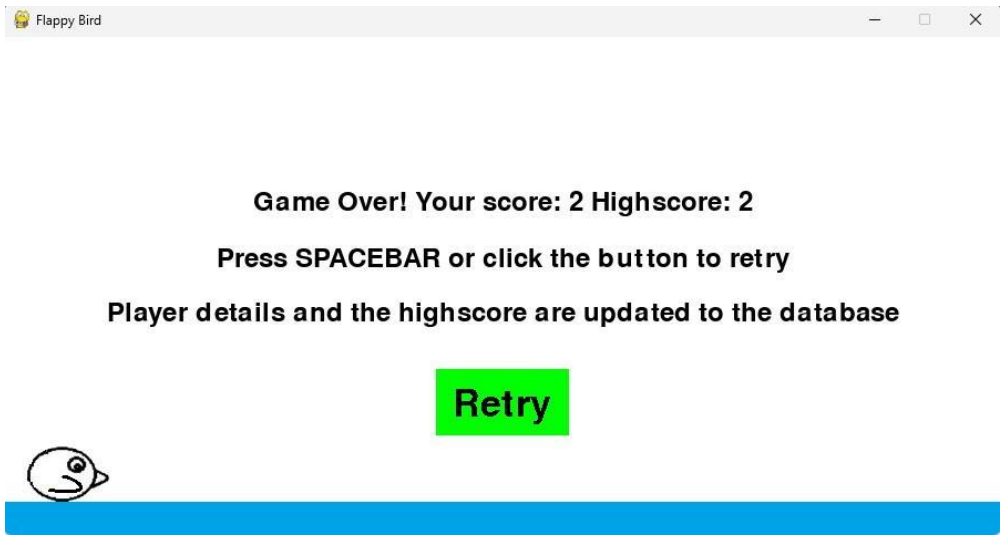
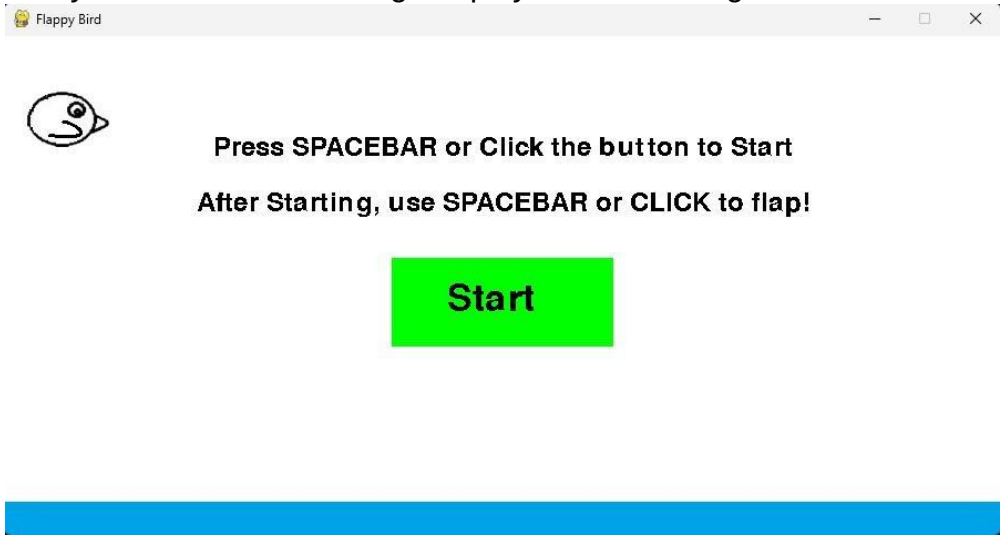
The game has a friendly user interface(UI), with clearly labelled buttons and instructions allowing the player to understand the mechanics. The UI elements are consistent in colour, font and size guiding the user to work with the interface accordingly. The screenshots in other test cases cover all of the user interfaces. The pictures below show the UI of the game. Game-menu Phase



Game-active phase

	 Game-over phase 
Test 23	The speed of the poles increases with each level, with each level advancing every 10 points in the score. However, there is a maximum speed limit to ensure the game remains playable. The screenshots show the change in value for speed_multiplier.

	 The image contains two screenshots of the Flappy Bird game. The top screenshot shows the game at Level 2 with a score of 12 and a speed of 1.1x. The bottom screenshot shows the game at Level 4 with a score of 30 and a speed of 1.3x. Both screenshots show a bird flying through green pipes over a blue ground.
Test 24	The death screen shows both the score and highscore

	
Test 25	The retry button exists, allowing the player to start the game at will. 

Results of persona testing

Persona Test Cases	Case Tested	Comment
1.Check if Bill has any trouble in the input validation phase to input his data.	✓	Bill had no issues with the user interface and could provide accurate data with no errors.
2.Check if Bill can read the instructions and error messages precisely and provide accurate data input.	✓	When Bill tried to input incorrect data into the game, the game didn't start because of issues with how the data was captured. In other words, the program couldn't properly process the wrong

		information Bill entered, which prevented the game from loading.
3. Check if Bill can recognize the game menu screen, read the in-game instructions and start playing the game.	✓	Bill had no issues in understanding the game logic and reading instructions with ease. The instructions were clear enough to let Bill know what he needs to do next. The option to use either spacebar or mouse button 1 was convenient for Bill.
4. Observe if Bill understands the game mechanics such as controlling the bird and avoiding the poles.	✓	Bill initially struggled to understand the gravity mechanics in Flappy Bird but learned to control the bird's flight after one or two retries. Most of his deaths resulted from collisions with the poles rather than hitting the ground.
5. Check if Bill can recognize the game-over screen and navigate to the retry button and retry the game successfully.	✓	Bill is able to recognize the game over screen and see his score and highscore after death. He is also able to navigate to the retry button easily.